

# WAREFLOW

Automatización de una nave logística de productos de alimentación



## Memoria técnica

**Código de equipo:** G1-04

**Integrantes:** Gabriel Arnedo Espinosa  
Pablo Gómez San Frutos  
Carla Castellanos Gómez  
Nuria Urrecho Torres

**Fecha:** 03/06/2026

**Contacto:** [nurrtor@etsinf.upv.es](mailto:nurrtor@etsinf.upv.es)

Grado en Informática Industrial y Robótica  
Memoria técnica de proyecto de automatización para PR3

---

# Índice general

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                    | <b>4</b>  |
| <b>1. Introducción</b>                                            | <b>5</b>  |
| 1.1. Contexto y justificación . . . . .                           | 5         |
| 1.2. Rol desempeñado en el proyecto . . . . .                     | 6         |
| 1.3. Descripción general del proceso logístico . . . . .          | 6         |
| <b>2. Propuesta de Automatización</b>                             | <b>8</b>  |
| 2.1. Análisis de alternativas . . . . .                           | 8         |
| 2.1.1. Alternativa 1: Automatización parcial . . . . .            | 8         |
| 2.1.2. Alternativa 2: Automatización integral del flujo . . . . . | 8         |
| 2.2. Justificación de la solución propuesta . . . . .             | 8         |
| 2.3. Objetivos del proyecto . . . . .                             | 9         |
| 2.3.1. Objetivo general . . . . .                                 | 9         |
| 2.3.2. Objetivos específicos . . . . .                            | 9         |
| 2.3.3. Descripción del proceso automatizado . . . . .             | 10        |
| 2.3.4. KPIs . . . . .                                             | 10        |
| 2.4. Impactos adicionales . . . . .                               | 11        |
| 2.5. Límites, condicionantes y restricciones . . . . .            | 11        |
| <b>3. Diseño de la Solución</b>                                   | <b>13</b> |
| 3.1. Elementos y dispositivos del sistema . . . . .               | 13        |
| 3.1.1. Estanterías . . . . .                                      | 13        |
| 3.1.2. Sistemas ASRS . . . . .                                    | 14        |
| 3.1.3. Colas . . . . .                                            | 14        |
| 3.1.4. Cintas transportadoras . . . . .                           | 14        |
| 3.1.5. Combiners . . . . .                                        | 14        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 3.1.6. Separators . . . . .                                     | 15        |
| 3.1.7. Robots . . . . .                                         | 15        |
| 3.1.8. AGVs . . . . .                                           | 15        |
| 3.1.9. Camiones . . . . .                                       | 15        |
| 3.2. Coordinación y comunicación entre actividades . . . . .    | 16        |
| 3.3. Distribución del espacio de trabajo . . . . .              | 16        |
| 3.4. Estrategias y algoritmos inteligentes utilizados . . . . . | 18        |
| 3.4.1. Asignación automática de carga a ASRS . . . . .          | 18        |
| 3.4.2. Actualización secuencial del inventario . . . . .        | 18        |
| 3.4.3. Generación probabilística de pedidos . . . . .           | 18        |
| 3.4.4. Control inteligente de conflictos entre AGVs . . . . .   | 19        |
| 3.5. Organización de la información . . . . .                   | 19        |
| 3.5.1. Tablas utilizadas . . . . .                              | 19        |
| 3.5.2. Grupos utilizados . . . . .                              | 19        |
| 3.6. Lógica de la planta a alto nivel . . . . .                 | 20        |
| 3.7. Descripción de la implementación en FlexSim . . . . .      | 21        |
| 3.7.1. Bloque de seco . . . . .                                 | 21        |
| 3.7.2. Bloque de fresco . . . . .                               | 23        |
| 3.7.3. Bloque carga camiones . . . . .                          | 26        |
| 3.7.4. Bloque de control de productos . . . . .                 | 26        |
| 3.7.5. Bloque operario . . . . .                                | 27        |
| <b>4. Pruebas y Validación</b>                                  | <b>28</b> |
| 4.1. Estrategia de pruebas y criterios de éxito . . . . .       | 28        |
| 4.2. Descripción general de la simulación . . . . .             | 28        |
| 4.3. Experimentación 1: política de recarga en seco . . . . .   | 29        |
| 4.4. Experimentación 2: política de recarga en fresco . . . . . | 33        |
| 4.5. Experimentación 3: número de combiners en seco . . . . .   | 35        |
| 4.6. Experimentación 4: AGVs y combiners en fresco . . . . .    | 38        |
| 4.7. Validación de resultados . . . . .                         | 42        |

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>5. Aspectos Avanzados de Control y Navegación</b>                       | <b>45</b> |
| 5.1. Sensorización e Integración . . . . .                                 | 45        |
| 5.1.1. Sensorización . . . . .                                             | 45        |
| 5.1.2. Integración . . . . .                                               | 48        |
| 5.2. Cruce operativo entre AGVs . . . . .                                  | 49        |
| 5.3. Aplicación del algoritmo A* . . . . .                                 | 51        |
| 5.4. Agentes Inteligentes utilizados . . . . .                             | 52        |
| 5.4.1. Agente de cruce . . . . .                                           | 52        |
| 5.4.2. Agente de camiones . . . . .                                        | 53        |
| 5.4.3. Agente supervisor . . . . .                                         | 55        |
| <b>6. Conclusiones y Recomendaciones</b>                                   | <b>57</b> |
| <b>Referencias</b>                                                         | <b>59</b> |
| <b>A. Anexos</b>                                                           | <b>60</b> |
| A.1. Manual de instalación y funcionamiento . . . . .                      | 60        |
| A.2. Descripción del script <code>lista_ordenes.py</code> . . . . .        | 60        |
| A.3. Descripción del script <code>lista_ordenes_fresco.py</code> . . . . . | 63        |
| A.4. Integración Python–FlexSim . . . . .                                  | 65        |
| <b>Agradecimientos</b>                                                     | <b>66</b> |

---

# Resumen

El proyecto desarrolla la propuesta de automatización integral de una nave logística destinada al abastecimiento de grandes superficies de alimentación. La instalación se divide en dos áreas operativas con condicionantes claramente diferenciados: una sección de producto seco a temperatura ambiente y una sección de producto fresco con requerimientos térmicos estrictos. La solución planteada combina sistemas automáticos de almacenamiento y recuperación (ASRS), transportadores, AGVs, robots de manipulación, estaciones de paletizado y lógica de control distribuida implementada en FlexSim.

La motivación principal del proyecto reside en la necesidad de sustituir operaciones manuales intensivas, repetitivas y con elevada carga ergonómica, especialmente en el montaje de pallets de gran altura y en la reposición de inventario. La automatización propuesta persigue simultáneamente cuatro objetivos: aumentar la capacidad de preparación de pedidos, mejorar la estabilidad física de los pallets expedidos, reducir los riesgos laborales y optimizar la gestión dinámica del inventario.

La memoria recoge el análisis funcional del sistema, la justificación de la solución adoptada, el diseño de la planta, la arquitectura lógica de control, la organización de los datos utilizados en la simulación y la validación de los principales parámetros de diseño mediante experimentación.

El trabajo ha permitido integrar conocimientos de automatización, simulación discreta, logística interna, control de flotas AGV y programación de apoyo mediante Python. Asimismo, ha puesto de manifiesto la importancia de equilibrar capacidad, robustez y complejidad operativa en el diseño de sistemas intralogísticos automatizados.

# Introducción

## 1.1. Contexto y justificación

El sector de la distribución alimentaria opera con márgenes ajustados, alta presión sobre los tiempos de entrega y una fuerte variabilidad en la demanda. En este contexto, las plataformas logísticas constituyen un elemento crítico de la cadena de suministro, ya que de su eficiencia depende tanto la disponibilidad de producto en tienda como la estabilidad operativa de la red de abastecimiento.

La preparación de pedidos en almacenes de alimentación presenta particularidades que dificultan su automatización completa. Por una parte, el producto seco concentra gran volumen de referencias, elevada rotación y recorridos intensivos de extracción. Por otra, el producto fresco exige mantener la cadena de frío, minimizar tiempos de exposición y coordinar flujos en espacios más restringidos. A ello se suma la necesidad de formar pallets de expedición estables, compactos y seguros para el transporte.

En numerosos entornos reales, una parte sustancial de estas tareas continúa realizándose de forma manual o semiautomática. Ello implica desplazamientos continuos, manipulación de cargas, apilado manual de cajas y dependencia de la pericia del operario para lograr pallets estables. Esta situación incrementa el riesgo ergonómico, reduce la repetibilidad del proceso y dificulta la estandarización del rendimiento.

La automatización propuesta en este proyecto responde a esa problemática mediante una concepción integral del sistema: almacenamiento automatizado, generación dinámica de pedidos, transporte interno coordinado, paletizado asistido por robots y control de inventario con lógica de recarga. El modelo desarrollado reproduce el comportamiento de una nave logística de gran distribución inspirada en plataformas reales del sector, adaptada a un enfoque académico y validada mediante simulación en FlexSim.

Más allá de la mejora puramente productiva, el interés del proyecto está en comprobar hasta qué punto una automatización bien planteada puede cambiar la forma de trabajar de la planta. No se trata solo de mover más cajas en menos tiempo, sino de conseguir un sistema más ordenado, más previsible y menos dependiente de decisiones improvisadas.

## 1.2. Rol desempeñado en el proyecto

El equipo adopta el papel de **consultora externa de ingeniería y automatización logística**. Desde esta perspectiva, el trabajo se orienta a diseñar una solución técnicamente viable, justificarla mediante criterios cuantificables y validar sus parámetros principales antes de una hipotética implantación industrial.

Este enfoque implica:

- analizar el proceso actual y detectar limitaciones operativas;
- proponer alternativas de automatización compatibles con los requisitos del negocio;
- diseñar la arquitectura física y lógica del sistema;
- simular su comportamiento para evaluar capacidad, estabilidad y robustez;
- formular recomendaciones finales fundamentadas en resultados.

## 1.3. Descripción general del proceso logístico

La instalación considerada abastece supermercados mediante la expedición de pallets completos correspondientes a pedidos de tienda. Cada pedido está compuesto por 12 líneas de producto. Las referencias se almacenan inicialmente en estanterías organizadas por familias y son extraídas según la demanda generada durante la simulación.

La planta se divide en dos grandes subsistemas:

- **Sección de seco:** zona a temperatura ambiente, con alta capacidad de almacenamiento y transporte interno mediante ASRS y cintas.
- **Sección de fresco:** zona refrigerada, donde el transporte entre almacenamiento y paletizado se realiza mediante AGVs a través de una red con esclusa y puntos críticos de cruce.

Además, se incorporan subsistemas auxiliares para la recarga de inventario, la descarga de pallets de reposición y la expedición final hacia camiones de distribución.

Desde el punto de vista operativo, la lógica general parte de una idea sencilla: extraer solo aquello que pide la demanda, agruparlo de forma ordenada y mantener el inventario en niveles suficientes sin disparar movimientos innecesarios de recarga. La dificultad real aparece al trasladar esa idea a dos entornos distintos, uno más lineal y otro más condicionado por restricciones térmicas y de circulación. Precisamente por eso la memoria

no plantea una solución única para toda la nave, sino una automatización ajustada al comportamiento específico de cada sección.



# Propuesta de Automatización

## 2.1. Análisis de alternativas

Antes de definir la solución final, se consideraron dos enfoques generales de automatización:

### 2.1.1. Alternativa 1: Automatización parcial

Se basa en automatizar almacenamiento y algunos transportes internos, manteniendo operaciones manuales o semimanuales en la consolidación final de pedidos. Esta solución mejora la productividad respecto al modelo tradicional, pero conserva cuellos de botella en el paletizado y no elimina por completo los problemas de seguridad y calidad del pallet.

### 2.1.2. Alternativa 2: Automatización integral del flujo

La segunda alternativa, adoptada en este proyecto, combina ASRS, cintas, AGVs, robots, combiners y lógica automática de recarga. Esta opción requiere mayor desarrollo inicial, pero permite atacar simultáneamente los principales problemas del sistema: manipulación manual, errores de secuenciación, baja robustez del inventario y limitada capacidad de expedición.

La comparación entre alternativas se realizó pensando no solo en la inversión inicial, sino también en el comportamiento esperado del sistema a medio plazo. La opción parcial supone una mejora real, aunque tiende a desplazar el cuello de botella hacia la consolidación final. Por eso la alternativa integral, aunque más ambiciosa, es la única que permite estudiar la planta como un sistema conectado de extremo a extremo.

## 2.2. Justificación de la solución propuesta

La solución seleccionada se justifica por las siguientes razones:

- reduce de forma sustancial la intervención manual en tareas repetitivas y de esfuerzo;

- aumenta la trazabilidad del flujo de materiales y del estado del inventario;
- permite adaptar la lógica de operación a dos entornos térmicos distintos;
- facilita la experimentación previa de parámetros críticos antes de una implantación real;
- mejora la estabilidad física de los pallets mediante paletizado automatizado y control geométrico;
- incrementa la resiliencia del sistema frente a variaciones de demanda mediante recarga dinámica.

La planta presenta además tolerancia a fallos en la sección de fresco. Dado que la asignación se realiza por pedido mediante *tokens* y cada orden conserva su `orderID`, la caída de un agente de transporte no bloquea la operativa completa: los AGVs restantes pueden seguir atendiendo las colas y completar los pedidos pendientes siempre que exista capacidad disponible en la red. En otras palabras, la lógica de control no depende de un único vehículo, sino de una asignación dinámica distribuida entre los agentes activos.

En términos prácticos, la propuesta se eligió porque es la que mejor se adapta al tipo de problema que plantea la nave. No tendría mucho sentido automatizar solo el almacenamiento si después la salida del pedido sigue dependiendo de operaciones lentas o difíciles de coordinar. Del mismo modo, tampoco resultaría eficiente automatizar el paletizado sin controlar antes la llegada ordenada de producto y la reposición de inventario. La solución final intenta precisamente evitar esas automatizaciones “a medias” que mejoran una parte del sistema, pero no resuelven el problema global.

## 2.3. Objetivos del proyecto

### 2.3.1. Objetivo general

Diseñar y validar mediante simulación una nave logística automatizada para productos de alimentación, capaz de preparar pedidos de forma segura, eficiente y trazable, diferenciando adecuadamente entre los flujos de producto seco y fresco.

### 2.3.2. Objetivos específicos

- Automatizar la extracción de producto desde estanterías mediante sistemas ASRS.
- Diseñar un sistema de transporte interno adaptado a cada entorno de operación.

- Implementar una lógica de paletizado que garantice la formación de pallets completos y estables.
- Incorporar un mecanismo automático de detección de necesidades de recarga.
- Dimensionar los principales recursos del sistema mediante experimentación en FlexSim.
- Validar la coordinación entre AGVs en zonas de conflicto y paso por esclusa.
- Integrar scripts externos para la generación dinámica de pedidos y el control del inventario.

### 2.3.3. Descripción del proceso automatizado

El proceso automatizado puede resumirse en las siguientes etapas:

1. Recepción o generación del pedido.
2. Identificación de los productos requeridos.
3. Extracción de unidades desde estanterías mediante ASRS.
4. Transporte hacia la zona de paletizado.
5. Agrupación y paletizado del pedido.
6. Verificación funcional del pallet generado.
7. Traslado del pallet expedido al camión de salida.
8. Monitorización del inventario y activación de recargas cuando corresponda.

### 2.3.4. KPIs

Para evaluar el rendimiento de la propuesta se definieron los siguientes indicadores:

- **Pallets completados:** número total de pallets terminados por unidad de tiempo.
- **Tiempo medio de formación de pallet:** duración media del paletizado.
- **Stock mínimo positivo:** evitar roturas de stock.
- **Aprovechamiento de los camiones:** pallets por viaje.
- **Cuellos de botella:** posibles zonas de acumulación de productos o pallets.

En la experimentación y validación de resultados se comprobará el cumplimiento de dichas metas.

## 2.4. Impactos adicionales

La propuesta genera impactos positivos en distintos ámbitos:

- **Operativo:** reducción de tiempos improductivos, mejora de secuenciación y mayor capacidad.
- **Logístico:** mejor control del inventario y mayor regularidad del flujo de expedición.
- **Ergonómico:** disminución de la manipulación manual de cargas.
- **Tecnológico:** integración de simulación, lógica de control, agentes inteligentes y apoyo mediante Python.
- **Calidad:** pallets más homogéneos, trazables y con menor probabilidad de inestabilidad.

También conviene matizar que no todos los impactos son inmediatos ni del mismo tipo. Algunos, como la reducción de manipulación manual o la mejora de la trazabilidad, aparecen desde el primer momento. Otros, como la mejora del rendimiento global o la reducción de ineficiencias logísticas, dependen más de un buen ajuste de parámetros y de una correcta coordinación entre recursos. Por eso la experimentación se ha planteado como una parte central del proyecto y no como una comprobación secundaria.

## 2.5. Límites, condicionantes y restricciones

La propuesta está sujeta a varias restricciones de diseño:

- mantenimiento estricto de las condiciones térmicas en la sección de fresco;
- capacidad limitada de los camiones de expedición, fijada en 9 pallets, valor calculado a partir de las dimensiones del pallet y del camión en FlexSim;
- necesidad de coordinar cruces y esclusas en la red de AGVs;
- dependencia del flujo de alimentación de los combiners respecto a la disponibilidad aguas arriba;
- simplificaciones propias de un modelo de simulación, que no incluyen todos los fenómenos físicos reales.

Además de estas restricciones, el propio hecho de trabajar con un modelo de simulación obliga a tomar algunas decisiones de simplificación. Se han representado de manera detallada los flujos, los recursos y los puntos de decisión que más afectan al rendimiento

de la planta, pero no se ha buscado reproducir todos los matices de una instalación real. El objetivo del proyecto no es replicar exactamente una nave existente, sino disponer de una base suficientemente realista para comparar alternativas y justificar decisiones de automatización.

## Diseño de la Solución

### 3.1. Elementos y dispositivos del sistema

La planta se ha estructurado en tres bloques: *seco*, *fresco* y *extra*. Los dos primeros representan las áreas principales de almacenamiento y preparación de pedidos. El bloque extra modela el proceso auxiliar de formación de pallets de reposición que alimentan nuevamente a la instalación.

Tabla 3.1: Distribución de dispositivos por secciones

| Elemento    | Seco | Fresco | Extra | Total |
|-------------|------|--------|-------|-------|
| Estanterías | 12   | 8      | 0     | 20    |
| ASRS        | 6    | 4      | 0     | 10    |
| Colas       | 13   | 10     | 4     | 27    |
| Cintas      | 2    | 0      | 0     | 2     |
| Combiners   | 3    | 1      | 2     | 6     |
| Separators  | 1    | 1      | 0     | 2     |
| Robots      | 4    | 2      | 2     | 8     |
| AGVs        | 2    | 9      | 2     | 13    |
| Camiones    | 4    | 2      | 2     | 8     |

#### 3.1.1. Estanterías

Constituyen el soporte principal de almacenamiento de producto. Su organización por familias y referencias permite asignar ubicaciones de forma estructurada, simplificando la lógica de extracción y de recarga.

En el modelo, las estanterías no son solo un elemento de almacenamiento estático. También actúan como referencia para la lógica de control, ya que la codificación de las familias de producto permite decidir de forma automática qué recurso debe intervenir y hacia qué parte del sistema tiene que dirigirse cada caja. Esta organización contribuye a que el comportamiento del almacén sea más ordenado y más fácil de escalar.

### **3.1.2. Sistemas ASRS**

Los sistemas automáticos de almacenamiento y recuperación realizan la extracción y deposición de cajas en las zonas intermedias del proceso. En el modelo se asignan a referencias concretas mediante reglas de clasificación derivadas del identificador de producto.

Su papel es especialmente importante porque marcan el ritmo al que el producto abandona el almacenamiento. Si esta etapa trabaja por debajo de la demanda, todo el sistema se resiente; si trabaja por encima sin coordinación suficiente, aparecen acumulaciones aguas abajo. Por eso la asignación de cargas al ASRS correspondiente se ha tratado como una decisión de control relevante y no como una simple conexión física dentro del modelo.

### **3.1.3. Colas**

Las colas actúan como elementos desacopladores entre operaciones. Su función es absorber pequeñas fluctuaciones de ritmo entre almacenamiento, transporte, paletizado y expedición, evitando bloqueos innecesarios y mejorando la continuidad del sistema.

En una planta real, este papel amortiguador es fundamental. Pocas veces todos los recursos trabajan exactamente al mismo ritmo, y pretender un acoplamiento rígido entre extracción, transporte y consolidación suele traducirse en bloqueos o tiempos muertos. En el modelo, las colas permiten representar ese margen operativo razonable que hace posible que el sistema siga funcionando aunque aparezcan pequeñas variaciones entre procesos.

### **3.1.4. Cintas transportadoras**

Se emplean exclusivamente en la sección de seco, donde el caudal de producto y la regularidad espacial favorecen una solución lineal de transporte. Durante el desarrollo del proyecto se comprobó que, para este subsistema, las cintas resultaban más eficientes y menos complejas que una solución equivalente basada íntegramente en AGVs.

### **3.1.5. Combiners**

Los combiners consolidan las 12 líneas de cada pedido en un único pallet de expedición. En una implantación real, esta estación incorporaría sensores y visión artificial para verificar altura, ocupación de huecos, alineación y estabilidad estructural del pallet.

El combiner es uno de los puntos más sensibles de la instalación porque ahí se materializa el valor logístico del proceso: pasar de unidades sueltas a un pallet terminado listo para expedición. Si esta estación queda corta de capacidad, la planta pierde rendimiento aunque

el resto de recursos funcionen bien. Si, por el contrario, se sobredimensiona, se incrementa el coste y la complejidad sin una mejora proporcional. Esa es precisamente una de las cuestiones que luego se contrasta en la experimentación.

### **3.1.6. Separators**

Se utilizan en el proceso inverso de reposición. Los pallets de recarga llegan con 12 unidades homogéneas y se despaletizan para reintroducir el material en la operativa de almacenamiento.

### **3.1.7. Robots**

Los robots de manipulación realizan tareas de transferencia entre colas, separators y combiners, especialmente en las zonas auxiliares de recarga y en operaciones donde se requiere precisión en el manejo del pallet.

### **3.1.8. AGVs**

Los AGVs asumen dos funciones diferenciadas:

- transporte de pallets completos entre colas de expedición y camiones;
- transporte de cajas en la red interna de fresco entre salidas de ASRS y entradas a combiner o separator.

Esta doble función explica por qué el comportamiento de los AGVs no tiene el mismo peso en seco y en fresco. En la zona de seco su papel está más concentrado en el último tramo hacia el camión. En fresco, en cambio, forman parte del transporte interno principal y condicionan directamente la alimentación de los combiners. Por eso el análisis de su número y de su interacción con la red es mucho más importante en esa sección.

### **3.1.9. Camiones**

Se contemplan tanto camiones de expedición a tienda como camiones de recarga de inventario. Los segundos están limitados a una capacidad máxima de 9 pallets por viaje ya que, teniendo en cuenta las dimensiones del camión y del pallet, se ha estimado que ese es la capacidad máxima del vehículo.



## 3.2. Coordinación y comunicación entre actividades

La coordinación del sistema se articula mediante tres mecanismos principales:

- **conectores físicos y lógicos**, que establecen la secuencia de transferencia entre objetos;
- **tokens de Process Flow**, empleados para transportar estado, atributos de pedido, identificadores de producto y referencias de recurso;
- **event-triggered sources**, utilizados para reaccionar automáticamente ante entradas o salidas relevantes del sistema.

Esta arquitectura permite desacoplar la lógica de control del comportamiento puramente geométrico de la planta, facilitando la trazabilidad de cada pedido y la sincronización entre subsistemas.

Desde un punto de vista práctico, esta forma de organizar la comunicación simplifica bastante el modelo. En lugar de depender de reglas aisladas dentro de cada objeto, gran parte del comportamiento se controla mediante tokens y eventos, lo que permite seguir con más claridad el estado de cada pedido, de cada referencia y de cada recurso. Esto también hace más sencillo modificar la lógica del sistema sin tener que rehacer toda la estructura física de la planta.

## 3.3. Distribución del espacio de trabajo

La disposición en planta se ha diseñado con criterios de separación funcional, continuidad de flujo y reducción de interferencias:

- la sección de seco agrupa almacenamiento, extracción y consolidación en una disposición más lineal, adecuada para el uso de cintas;
- la sección de fresco incorpora una red de AGVs y una esclusa, lo que obliga a una disposición más compacta y con control explícito de prioridades;
- las zonas extra de recarga se sitúan como subsistemas auxiliares, conectados lógicamente a los almacenes principales pero segregados del flujo ordinario de expedición;
- las áreas de carga y descarga de camiones se ubican en la periferia operativa para reducir recorridos de entrada y salida de pallets.

Desde el punto de vista industrial, esta distribución persigue minimizar recorridos cruzados, aislar la zona de fresco del tráfico no imprescindible y permitir que los flujos de reposición y expedición convivan sin generar bloqueos estructurales. A continuación se muestra el layout de la planta:

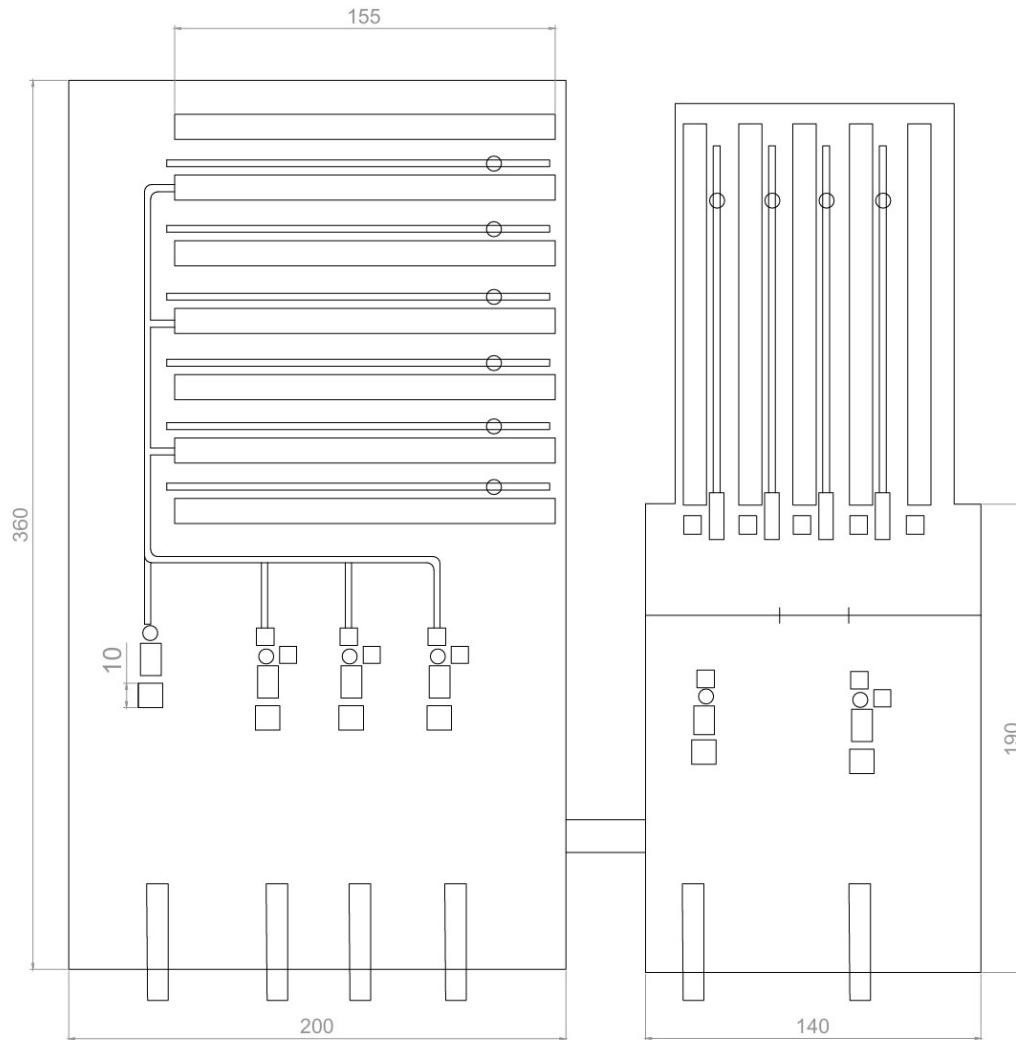


Figura 3.1: Layout de la nave.

Aunque la memoria no se centra en una optimización geométrica al detalle, sí se ha intentado que la distribución en planta tenga sentido desde el punto de vista de operación. La zona de seco se beneficia de una organización más lineal y más sencilla. La de fresco, por sus condicionantes, requiere más atención al movimiento y a la convivencia entre recursos. En ese sentido, la disposición elegida busca que el comportamiento del modelo no sea solo técnicamente correcto, sino también creíble desde una perspectiva logística.

## **3.4. Estrategias y algoritmos inteligentes utilizados**

### **3.4.1. Asignación automática de carga a ASRS**

Se ha implementado una lógica de selección basada en reglas que asigna cada carga al ASRS correspondiente en función del código del producto. El algoritmo recorre las zonas de carga disponibles y selecciona la primera compatible con la familia almacenada en el recurso.

Esta estrategia ofrece tres ventajas:

- elimina decisiones manuales de asignación;
- reduce errores de clasificación;
- evita búsquedas innecesarias al detenerse en la primera coincidencia válida.

### **3.4.2. Actualización secuencial del inventario**

Cada vez que una referencia se extrae del almacén, el sistema recorre la tabla de stock hasta localizar la fila correspondiente y descuenta automáticamente la unidad servida. Esta lógica garantiza la coherencia entre flujo físico y registro de inventario en tiempo real.

Se trata de una solución relativamente simple, pero suficiente para el alcance del modelo. No se ha buscado introducir un algoritmo complejo de gestión de inventario, sino asegurar que la simulación refleje de forma consistente el consumo real de producto. Ese punto era importante porque toda la lógica de recarga depende de que el stock registrado tenga sentido.

### **3.4.3. Generación probabilística de pedidos**

La demanda se ha modelado mediante scripts Python externos que generan órdenes dinámicas con 12 líneas por pedido. La selección de referencias se realiza con ponderaciones distintas para seco y fresco, reproduciendo perfiles de consumo diferenciados y evitando repeticiones dentro de una misma orden.

Esto aporta bastante realismo a la simulación. Si los pedidos fueran siempre iguales o estuvieran contruidos de forma totalmente uniforme, muchos de los cuellos de botella del sistema quedarían disimulados. Al introducir variabilidad y distintos pesos de aparición de las referencias, el modelo se comporta de una forma más cercana a lo que cabría esperar en una operación real.

### 3.4.4. Control inteligente de conflictos entre AGVs

En la red de fresco se han incorporado agentes de proximidad y áreas de control para evitar colisiones, regular velocidades y proteger la circulación del operario supervisor. Estas funcionalidades no persiguen únicamente la evitación del bloqueo, sino también la reproducción de un comportamiento operativo realista.

## 3.5. Organización de la información

### 3.5.1. Tablas utilizadas

- **Ordenes\_seco:** almacena identificador de pedido, producto y cola destino para la sección de seco.
- **Ordenes\_fresco:** análoga a la anterior, adaptada a la zona refrigerada.
- **Productos\_seco:** controla el inventario remanente por referencia en seco.
- **Productos\_fresco:** controla el inventario remanente por referencia en fresco.
- **Resultados\_combiners:** recopila resultados de experimentación relativos al número de combiners en seco.
- **Resultados\_combiners\_fresco:** recoge los resultados combinados de AGVs y combiners en fresco.

El uso de estas tablas no responde únicamente a una necesidad de almacenamiento de datos, sino también a una decisión de diseño del modelo. Centralizar la información principal en tablas facilita el seguimiento del inventario, la revisión de resultados y la validación de la lógica implementada. Además, hace que la simulación sea más legible cuando se quiere comprobar por qué se ha producido una determinada decisión o un determinado flujo.

### 3.5.2. Grupos utilizados

- **AGVs\_fresco:** flota interna encargada del transporte entre colas en fresco.
- **Trucks:** conjunto de camiones de expedición y soporte.
- **Camiones\_carga:** grupo de los camiones que se reabastecen.
- **AGVs\_carga:** grupo de AGVs que atienden a los camiones que reabastecen.

- **ASRS\_seco y ASRS\_fresco:** facilitan la selección dinámica del recurso de extracción.
- **Queue\_pallets:** agrupa las colas donde quedan depositados los pallets completados.
- **AGV\_camion:** reúne los AGVs dedicados al flujo pallet-camión.
- **Est\_seco y Est\_fresco:** permiten enlazar la lógica física de salida con el control del inventario.

Los grupos cumplen una función parecida desde el punto de vista del control de recursos. Permiten trabajar con conjuntos homogéneos de objetos sin tener que codificar una lógica distinta para cada elemento individual. Esto hace que el modelo sea más manejable y evita duplicar código o reglas de comportamiento en demasiados puntos distintos.

### 3.6. Lógica de la planta a alto nivel

La lógica de la planta se ejecuta en el ProcessFlow de Flexsim:

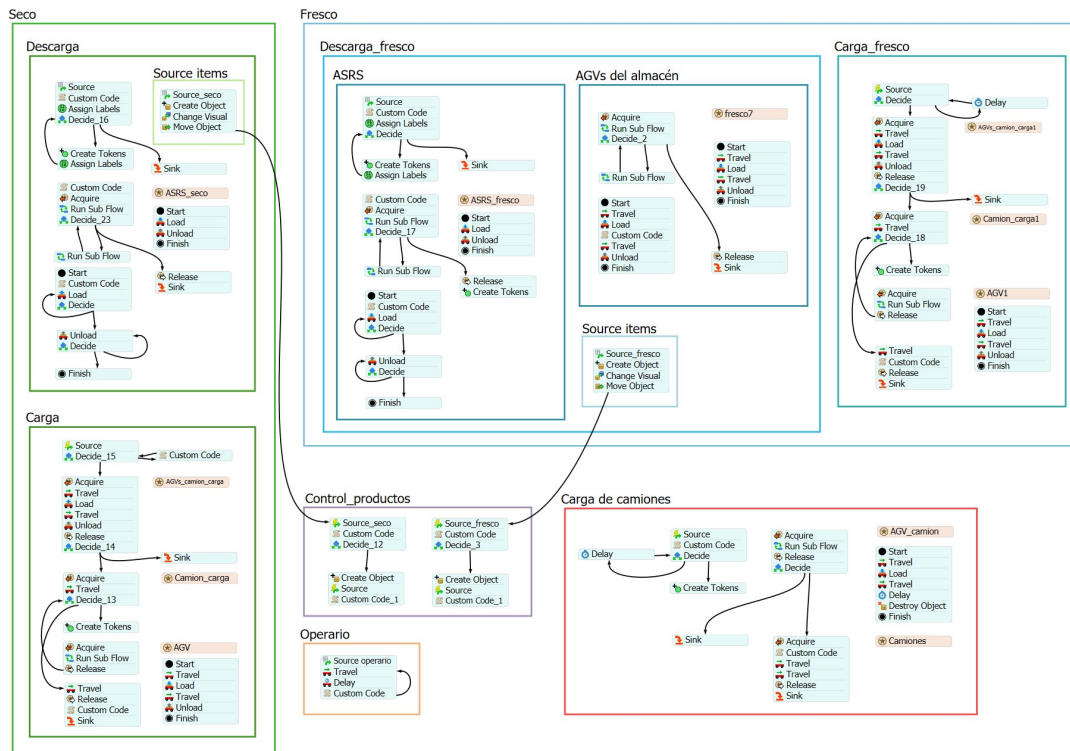


Figura 3.2: ProcessFlow del modelo.

Traducido en un diagrama de flujo:

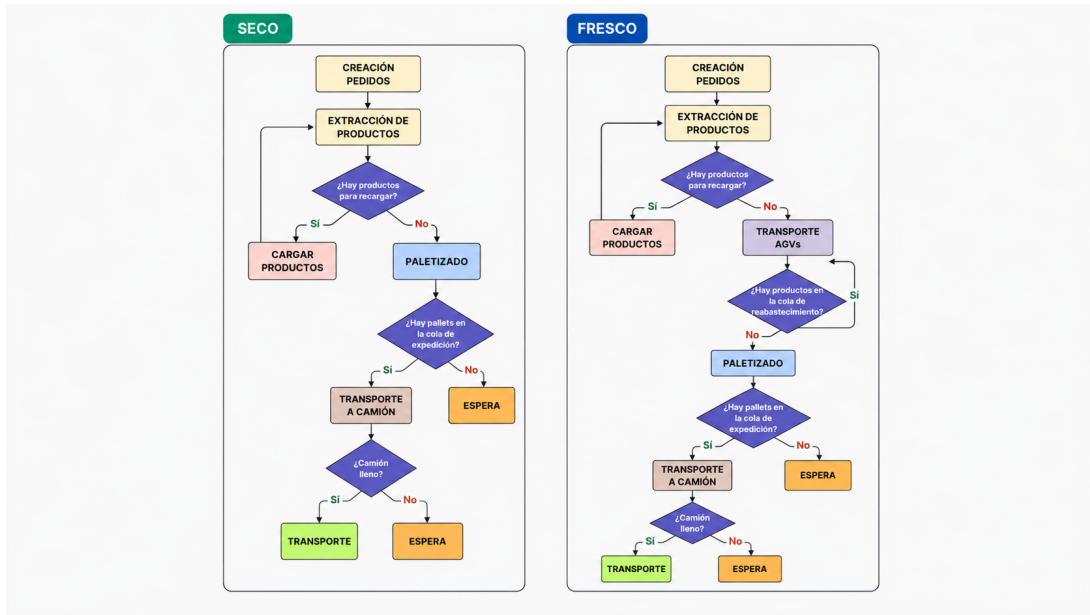


Figura 3.3: Diagrama de flujo del modelo.

## 3.7. Descripción de la implementación en FlexSim

### 3.7.1. Bloque de seco

La implementación de la sección de seco se estructuró en tres partes principales: la creación inicial de producto, la descarga de referencias desde almacén para preparar pedidos y la carga de pallets una vez terminados. Aunque a nivel general el flujo es relativamente lineal, a nivel de process flow fue necesario detallar cada operación para evitar errores de sincronización entre ASRS, cintas, colas y recursos móviles.

#### Sección source

En primer lugar, en el *Schedule Source* se crean 40 unidades de cada uno de los 120 productos de seco. A cada unidad se le asigna, por orden de lista, el valor correspondiente en el label `seco`. Después, en *Create Object*, se crea cada caja, se asigna el objeto a `token.source` y se generan los tokens asociados `seco` y `orderID`. El token `seco` toma el valor definido previamente en el source y `orderID` se inicializa a 0.

A continuación, en función del valor de `token.seco`, se utiliza *Change Visual* para dar a cada caja el color de la paleta correspondiente. Finalmente, en *Move Object* se obtiene la decena del valor de `token.seco` para localizar la estantería que corresponde a cada ítem y devolver ese *pointer*, de forma que cada producto queda almacenado en su posición inicial.

## Sección descarga

La descarga comienza con un *Schedule Source* con un *Offset Time* de 5 segundos, dejando así tiempo suficiente para que se hayan creado previamente todos los objetos del inventario inicial. Después se ejecuta un *Custom Code* en el que, mediante las funciones definidas en el archivo Python `lista_ordenes.py`, se crean las tablas `Ordenes_seco` y `Productos_seco`. De esta forma, la tabla `Ordenes_seco` recoge las órdenes, los productos y la cola final a la que deberá ir cada referencia, mientras que `Productos_seco` almacena las unidades disponibles de cada producto, que inicialmente son 40.

Seguidamente, en *Assign Labels* se inicializa `token.currentRow` a 1, valor que permite gestionar la fila actual de la tabla, y se calcula `token.totalRows` aplicando la función `.numRows` sobre `Ordenes_seco`. A continuación se ejecuta un *Conditional Decide* que comprueba si la fila actual es menor que `totalRows`. Si la condición no se cumple, el flujo finaliza en un *Sink*; si se cumple, se pasa a *Create Tokens*, donde se crea un token por cada fila de la tabla `Ordenes_seco`.

Después de este bloque hay un nuevo *Assign Labels* que devuelve el flujo al *Decide* y cuya función es sumar una unidad a `token.currentRow` para pasar a la fila siguiente. El token recién creado se dirige al bloque inferior, donde un *Custom Code* calcula `token.estanteria` y `token.asrs`, devolviendo la estantería y el número de ASRS que corresponde en función del label `seco`. Además, según el valor de `token.asrs`, se asigna `token.colaASRS`, es decir, el número del *EntryTransfer* de la *conveyor* donde el ASRS deberá dejar el objeto.

A continuación se realiza un *Acquire* del grupo `ASRS_seco`, de forma que se toma el ASRS cuya label `pedido`, asignada manualmente, coincida con `puller.asrs`. Así, cada ASRS trabaja únicamente con el rango de estanterías que le corresponde. Después se ejecuta un *Run Subflow* con un *load* y un *unload*.

En el *load*, en primer lugar, se resta una unidad del producto que se va a coger en la tabla `Productos_seco`. Después se asignan a los ítems labels importantes procedentes de los tokens, como `seco`, `orderId`, `colaDest` y `currentRow`. En el *Unload*, simplemente se descarga el primer ítem que tenga el ASRS en la cinta correspondiente.

Después de este subflujo se ejecuta un *Decide* que comprueba si hay al menos 12 ítems con el mismo `orderId` en alguna cola de recarga de almacén. Si no se cumple, se realiza un *Release* y un *Sink* para liberar el recurso. Si sí se cumple, se ejecuta un *Run Sub Flow* para recargar estanterías con esos ítems.

Dentro de este segundo subflujo, un *Custom Code* inicializa `token.cargados` a 0. Después se realiza el *load* en la cola correspondiente y se suma una unidad a ese token. Un *Decide* comprueba si ya se han cargado las 12 unidades o si hay que seguir cargando. Una vez

alcanzadas las 12, se pasa a *Unload*, donde se repite el mismo planteamiento: se descarga una unidad y un *Decide* verifica si ya se han descargado todas hasta llegar a 0.

## Sección carga

La sección de carga comienza con un *Event-Triggered Source* que se activa cuando aparecen pallets ya montados en *Queue50*. En ese momento se ejecuta un *Decide* para gestionar correctamente el flujo, ya que al montarse varios pallets de forma sucesiva aparecían errores si no se controlaba la secuencia. En este *Decide* se comprueba que haya algún pallet en la cola y que el camión esté listo para cargarse. Si no se cumplen ambas condiciones, se ejecuta un *Delay* de 100 segundos; si sí se cumplen, se hace un *Acquire* del AGV encargado de cargar el camión.

Ese AGV ejecuta entonces un bloque de *Travel*, *Load* y *Unload*. En cada iteración se suma una unidad al label **carga** del camión. Después se llega a un *Decide* en el que se comprueba si se ha alcanzado la variable global **capacidad**, que indica la capacidad máxima del camión. Si se alcanza, el camión realiza un *Travel* hasta el hueco del camión de carga situado dentro de la nave.

Una vez allí, se comprueba si la carga del camión es 0. Si no lo es, que es la situación habitual cuando acaba de llegar, se realiza un *Acquire* del AGV de descarga del camión dentro de la nave. Ese recurso ejecuta un *Run Sub Flow* en el que hace iteraciones de *Travel*, *Load* y *Unload*. En cada *load* se resta una unidad del label **carga** del camión. Al finalizar cada iteración, se hace un *Release* del recurso y se vuelve al *Decide* para comprobar de nuevo si la carga ya es 0. En caso de haber descargado completamente el camión, este vuelve a su posición inicial con un *Travel*, se libera el recurso mediante *Release* y se indica en el label **colocado** que el camión vuelve a estar aparcado y listo para cargarse otra vez.

### 3.7.2. Bloque de fresco

La lógica del fresco mantiene la misma filosofía general que la del seco, aunque introduce una mayor complejidad por el uso de AGVs en el transporte interno y por la necesidad de gestionar recargas y expediciones en una red móvil más sensible a bloqueos.

## Sección descarga: subsección source

La subsección *Source* del fresco sigue prácticamente la misma estructura que en seco. En el *Schedule Source* se generan 40 unidades de cada uno de los 80 productos y se les asigna el valor del label **fresco**. Después, en *Create Object*, se crean las cajas, se les



asigna `token.fresco`, se cambia su color con *Change Visual* y se mueven a la estantería correspondiente en función de ese valor, generando también `token.estanteria`.

### Sección descarga: subsección ASRS

Esta sección también es muy parecida a la del seco. En primer lugar hay un *Source* con *offset* de 5 segundos para dar tiempo a que se creen los objetos. Después, en un *Custom Code*, se rellenan las tablas `Productos_fresco` y `Ordenes_fresco` mediante las funciones del script `lista_ordenes_fresco.py`.

Una vez creadas las tablas, en *Assign Labels* se inicializa `token.currentRow` a 1 y se calcula `token.totalRows` a partir del número de filas de `Ordenes_fresco`. Después, un *Decide* comprueba que `currentRow` no sea mayor que `totalRows`, es decir, que todavía quedan productos por recoger. Si la condición se cumple, se crea un token para esa fila y dicho token se envía al *Custom Code* inferior. Una vez creado, en un nuevo *Assign Labels* se suma una unidad a `token.currentRow` para pasar al producto siguiente.

En el *Custom Code* de esta rama se determinan, en función de la centena y la decena de `token.fresco`, tanto la estantería como el ASRS que debe atender la orden. Además, según el valor de `token.asrs`, también se define `token.colaASRS`, que indica en qué cola deberá dejar el objeto el ASRS. A continuación se realiza un *Acquire* del grupo `ASRS_fresco` y se selecciona el ASRS cuyo label `pedido`, definido previamente de forma manual, coincida con `token.asrs`. Después se ejecuta un *RunSubflow* con un *load* y un *unload* de la caja.

En el *load* se descuenta una unidad del producto correspondiente en la tabla `Productos_fresco`, lo que permite mantener actualizado el inventario. Después se copian a los *flowitems* algunos valores de los tokens, como `fresco`, `orderID` y `colaDest`, que luego serán necesarios. Una vez terminado el subflujo, se entra en un *Decide* que comprueba si hay 12 ítems del mismo `orderID` en alguna de las colas de carga destinadas a recargar estanterías.

Si no se cumple la condición, simplemente se hace un *Release* del recurso y se crea un token para que los AGVs recojan la caja que acaba de dejarse. Si sí se cumple, entonces se ejecuta un *RunSubflow* en el que, en primer lugar, un *Custom Code* pone `token.cargados` a 0 para controlar la cantidad de ítems cargados por el ASRS. Después se hace un *load* y un *Decide*. Cuando `token.cargados` es mayor o igual que 12, el token vuelve a ponerse a 0 y comienza la fase de *Unload*, que continúa hasta descargar los 12 ítems.

### Sección descarga: subsección AGVs

Esta subsección se ejecuta después del *Create Tokens* anterior y se ocupa del transporte de las cajas hasta las colas de los combiners, donde se paletizan los pedidos. Para ello, primero

se realiza un *Acquire* del grupo **AGVs\_fresco**, formado por 7 AGVs. A estos vehículos se les ha asignado previamente la label **cola**: tres AGVs tienen valor 1 y los otros tres valor 2. En el *Acquire* se selecciona el AGV cuya label **cola** coincide con **token.colaASRS**.

Después se ejecuta un *Run Sub Flow* en el que el AGV viaja hasta la cola donde se ha depositado la caja, carga la caja con menor **orderID** para respetar el orden de los pedidos, viaja a la cola del combiner correspondiente según la tabla **Ordenes\_fresco** y descarga allí la caja.

Al finalizar, un *Decide* comprueba si hay algún ítem en la cola donde se dejan las cajas destinadas a recargar los almacenes. Si no lo hay, simplemente se libera el recurso y se termina en *Sink*. Si sí lo hay, se ejecuta otro *Run Sub Flow* en el que el AGV va a esa cola y carga el primer ítem disponible. Después, en un *Custom Code*, se asigna en función del label del ítem la cola de recarga a la que deberá dirigirse, guardándola en **token.cola\_carga**. El AGV viaja entonces a esa cola y descarga el ítem. Como este subflujo vuelve a conectar con el *Decide* anterior, si detecta que sigue habiendo cajas pendientes de recarga, vuelve a por ellas. De esta manera, el sistema da prioridad a la recarga del almacén.

## Sección carga

La sección de carga en fresco tiene lugar fuera de las naves principales. Comienza con un *Event Triggered Source* que se activa cuando llega un pallet a la cola 57. En el *Decide* se comprueba que en la cola haya al menos un ítem y que el camión esté colocado y listo para ser cargado. Si no se cumple, se ejecuta un *Delay* de 40 segundos y se vuelve a comprobar. Si sí se cumple, se toma el AGV como recurso, este carga el pallet de la cola, suma una unidad al label **carga** del camión y lo descarga en él. Después se hace un *Release* y se comprueba si la label **carga** del camión es mayor o igual que la capacidad.

Si no se cumple, el flujo termina y no se vuelve a ejecutar la secuencia hasta que entre otro pallet en la cola. Si se cumple, entonces se toma el camión con un *Acquire*, se pone su label **colocado** a 0 y se realiza un *Travel* hasta la zona de aparcamiento de camiones en la nave.

Una vez allí, se comprueba si la carga del camión es 0. Como inicialmente todavía no se ha descargado nada, se pasa a *Create Tokens*, donde se toma el AGV de carga de la nave mediante *Acquire* y se ejecuta un subflujo. En ese subflujo el AGV acude al camión y, en el *Load*, resta una unidad al label **carga** del camión. Después viaja a la cola del *separator* y deja allí el pallet con *Unload*. Al acabar el subflujo, se libera el recurso y se vuelve a comprobar en el *Decide* si la carga del camión ya es 0. Cuando el AGV ha descargado por completo el contenido del camión, este realiza un nuevo *Travel* hasta la zona donde se crean los pallets; en un *Custom Code* se vuelve a poner el label **colocado** a 1, se libera

el camión y el flujo termina en *Sink*.

### 3.7.3. Bloque carga camiones

Este bloque gestiona la carga de los camiones desde el momento en que aparecen pallets ya montados en las colas de los combiners. Se parte de un *Event Triggered Source* que se activa *On Entry* en cualquiera de las colas del grupo `Queue_pallets`. Después se ejecuta un *Custom Code* que comprueba que realmente hay un pallet disponible para evitar errores. Además, se toman los datos del pallet y se guardan en distintos tokens, como `token.cola`, `token.pallet` y el valor que identifica si el pallet es de seco o de fresco.

A continuación se ejecuta un *Decide* que comprueba que la cola no esté vacía. Si lo esté, se hace un *Delay* y se vuelve al *Decide*. Si no lo esté, se realiza un *Create Tokens* con destino un *Acquire*. En ese *Acquire* se toma el grupo `AGV_camion` y se selecciona el AGV cuyo label `cola_camion`, definido previamente de forma manual, coincida con el valor asociado al pallet. Después se ejecuta un *Run Sub Flow*.

En ese subflujo el AGV va a la cola, recoge el pallet, suma una unidad al label `carga` del camión y viaja hasta el camión correspondiente, situado frente a la cola. Después se realiza un *Delay* de 3 segundos y se destruye el objeto. Esta última parte es solo visual, para que la simulación tenga sentido desde el punto de vista gráfico.

Al terminar el subflujo se libera el recurso y se ejecuta un *Decide* que comprueba si el label `carga` del camión es mayor o igual que la capacidad, es decir, si el camión esté lleno. Si no lo esté, el flujo termina y queda a la espera de que entre otro pallet en el *Triggered Source*. Si sí lo esté, entonces se adquiere el grupo `Trucks`. El camión se selecciona en función de si su label `camion` coincide con el valor de `token.pallet`. Después, en un *Custom Code*, se vuelve a poner el label `carga` del camión a 0 y el vehículo realiza una ida y vuelta mediante dos *Travel* hasta una cola situada a cierta distancia, simulando así el transporte de los pallets a tienda. Finalmente, el camión se libera con *Release* y el flujo termina en *Sink*.

### 3.7.4. Bloque de control de productos

Este bloque controla la generación de nuevos productos para recargar el almacén. Se han implementado dos estructuras prácticamente iguales, una para seco y otra para fresco, ambas enlazadas con su correspondiente bloque de *Source* de ítems.

Con un *Event Triggered Source* se controla cada vez que sale un ítem de alguna de las estanterías de los grupos `Est_Seco` y `Est_fresco`. Después, en un *Custom Code*, se con-

trola la segunda columna de las tablas de productos de seco y fresco. En función de la cantidad restante de cada referencia, se guarda en `token.recarga` el valor del producto que necesita recargarse y se coloca un 1 en la columna de “avisado” de la tabla. Los valores umbral de recarga se han asignado en función de la demanda de cada producto.

A continuación, un *Decide* recorre la tabla completa y comprueba si hay algún producto marcado como avisado, pero todavía no enviado. En ese caso se ejecuta *Create Object*. En este bloque se crean 12 cajas del producto en la cola 45 o 55, según se trate de seco o fresco. Además, se asignan los tokens de `seco` o `fresco` con el valor de `token.recarga` y se asigna el token `tipo`, con valor 1 para los productos secos y valor 2 para los frescos.

Después de esto, un robot y un combiner paletizan automáticamente las 12 unidades de cada producto y las dejan en una cola. En ese momento se activa el *Triggered Source* de las colas 50 o 57, según corresponda, y se ejecuta un *Custom Code* en el que se asigna a cada ítem el label `seco/fresco` y el label `rack` en función de la estantería a la que irán destinados.

### 3.7.5. Bloque operario

Este bloque es sencillo y se utiliza para representar el movimiento cíclico del operario de supervisión a través de distintos *network nodes* de la planta. Mediante un *Schedule Source*, el operario inicia su recorrido a los 20 segundos de simulación. A partir de ahí realiza un *Travel* hasta el siguiente nodo, un *Delay* de 60 segundos para simular que se detiene a observar la nave y, a continuación, se ejecuta un *Custom Code*.

En ese código se comprueba si el operario ha llegado al último nodo del recorrido. Si ya ha llegado, vuelve de forma regresiva al inicio; si no, continúa avanzando sumando una unidad a la variable global `contador_paseo`. Después se vuelve a ejecutar el *Travel* y el proceso continúa indefinidamente.

# Pruebas y Validación

## 4.1. Estrategia de pruebas y criterios de éxito

La validación se ha realizado mediante experimentación controlada en FlexSim, utilizando réplicas múltiples y comparación de escenarios parametrizados. En lugar de limitarse a comprobar si el modelo funcionaba, se planteó una batería de pruebas orientada a responder preguntas de diseño concretas: cuánta recarga conviene acumular antes de llamar al camión, cuántas líneas de paletizado necesita la zona de seco y qué combinación de AGVs y capacidad de combiner resulta más adecuada para la nave de fresco.

La idea de esta fase no era únicamente obtener “el mejor número”, sino entender cómo cambia el comportamiento de la planta cuando se fuerzan configuraciones más conservadoras o más exigentes. Por eso, en los distintos experimentos se compararon escenarios bajos, intermedios y altos, observando tanto la producción conseguida como la estabilidad del sistema entre réplicas.

Los criterios de éxito empleados han sido los siguientes:

- ausencia de inventarios negativos;
- mantenimiento de niveles de stock alejados de la rotura;
- capacidad de expedición elevada y estable;
- aprovechamiento suficiente de los recursos de transporte;
- ausencia de sobredimensionamiento no justificado;
- coherencia operativa del flujo entre almacenamiento, transporte y paletizado.

## 4.2. Descripción general de la simulación

La simulación representa una nave logística con dos secciones operativas, generación aleatoria de pedidos, flujo bidireccional de reposición y expedición, y control dinámico de los recursos compartidos. La duración de referencia para los experimentos principales ha sido de 20 000 segundos de producción, con 5 réplicas por escenario en los casos descritos.

En la zona de seco, el flujo se apoya principalmente en estanterías, ASRS, cintas y combiners. En la zona de fresco, en cambio, la operativa depende mucho más de la coordinación entre almacenamiento, red de AGVs y paso por esclusa. Esta diferencia hace que los resultados de ambas zonas no se puedan interpretar del mismo modo: en seco, los cuellos de botella aparecen de forma más directa en las estaciones de paletizado, mientras que en fresco el transporte tiene un peso decisivo sobre la capacidad final.

Para cada experimento se definieron medidas de rendimiento específicas. En general, se priorizaron indicadores sencillos de interpretar, directamente relacionados con el comportamiento logístico del sistema y fáciles de comparar entre escenarios. Cuando una métrica presentó valores anómalos, como ocurrió con algunos tiempos medios de paletizado, se utilizó solo como apoyo y no como criterio final de decisión.

### 4.3. Experimentación 1: política de recarga en seco

En esta experimentación se analiza la política de recarga de la sección de seco, estudiando cómo afecta la cantidad mínima de pallets necesaria para solicitar el camión de reposición. El objetivo es determinar qué configuración permite aprovechar adecuadamente el camión de recarga sin provocar niveles de inventario demasiado bajos.

La planta dispone de una lógica de inventario en la que cada producto tiene un umbral mínimo de stock definido en función de su demanda. Cuando varios productos caen por debajo de su umbral, se solicita un camión de recarga que trae pallets de reposición. Por tanto, el parámetro analizado influye directamente en el momento en el que se activa la recarga y en la cantidad de producto que llega a la planta. Se han comparado cuatro escenarios, correspondientes a distintos niveles de agrupación de la recarga:

| Escenario | Opción  | Valor |
|-----------|---------|-------|
| 1         | Bajo    | 3     |
| 2         | Medio   | 5     |
| 3         | Notable | 7     |
| 4         | Alto    | 9     |

Figura 4.1: Escenarios analizados para la recarga en seco.

Para evaluar los escenarios se han utilizado dos medidas principales:

- `inputsCola`: número de entradas de pallets de recarga al sistema;
- `minimoProductos`: mínimo stock alcanzado por cualquier referencia

La métrica `inputsCola` permite analizar el volumen de recarga que recibe el sistema. La métrica `minimoProductos` es la más importante desde el punto de vista de seguridad

del inventario, ya que indica hasta qué punto se reduce el stock antes de que la reposición consiga mantener el sistema estable. El primer gráfico muestra el número de entradas en la cola de recarga para cada escenario. Esta medida permite evaluar cuánta actividad de reposición genera cada política.

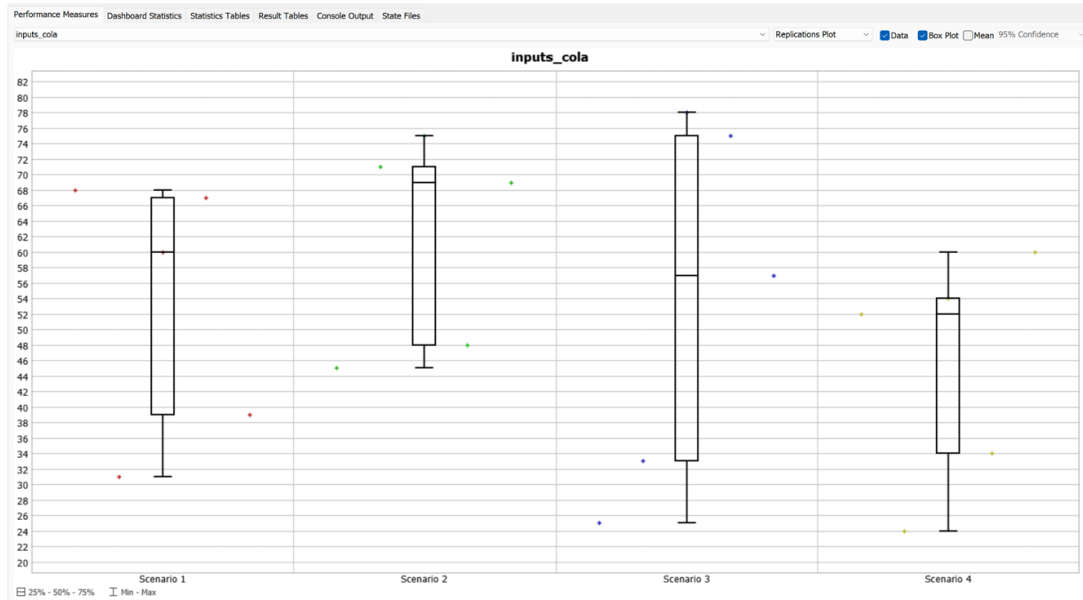


Figura 4.2: Resultados de `inputsCola` para los escenarios de recarga en seco.

En el Escenario 1, la mediana se sitúa aproximadamente en torno a 60 entradas. Sin embargo, existe una dispersión bastante elevada, con valores que van aproximadamente desde 31 hasta 68 entradas. Esto indica que la política baja genera un volumen de recarga considerable, pero con variabilidad entre réplicas. Al tratarse de una política más reactiva, el camión se solicita antes, lo que permite mantener una actividad frecuente de reposición.

En el Escenario 2, se observa la mediana más alta, aproximadamente alrededor de 69 entradas. Además, gran parte de los resultados se concentran en valores elevados, entre unas 48 y 71 entradas, llegando a máximos cercanos a 75. Esto significa que la política media genera una actividad de recarga intensa. Desde el punto de vista operativo, este escenario consigue alimentar con bastante frecuencia el sistema de reposición, aunque esta mayor actividad también puede implicar más trabajo para los recursos asociados a la descarga, transporte y almacenamiento.

En el Escenario 3, la mediana se sitúa aproximadamente alrededor de 57 entradas. No obstante, es el escenario con mayor dispersión, con valores que van desde unos 25 hasta aproximadamente 78. Esto indica que la política notable es más dependiente de la evolución concreta de la demanda en cada réplica. En algunas simulaciones se generan muchas entradas de recarga, mientras que en otras el sistema espera más tiempo antes de activar el camión. Esta variabilidad es lógica, ya que al exigir una mayor agrupación de productos

críticos, la llegada del camión depende más de que se acumulen suficientes necesidades de reposición.

En el Escenario 4, la mediana se sitúa alrededor de 52 entradas, siendo inferior a la de los escenarios anteriores. Los valores se mueven aproximadamente entre 24 y 60 entradas. Esto refleja que la política alta reduce la frecuencia o volumen de entradas de recarga. Al esperar más para solicitar el camión, se aprovecha mejor la agrupación, pero se genera menor actividad de reposición durante la simulación.

Desde el punto de vista de `inputsCola`, los escenarios 1 y 2 generan mayor actividad de recarga, mientras que los escenarios 3 y 4 tienden a agrupar más la reposición. Sin embargo, esta métrica no debe analizarse de forma aislada, ya que tener más entradas de recarga no implica necesariamente una política mejor. Lo importante es comprobar si esa política mantiene el inventario en niveles seguros.

El segundo gráfico muestra el valor mínimo de stock alcanzado por cualquier producto durante la simulación. Esta medida es fundamental para validar la seguridad de la política de recarga.

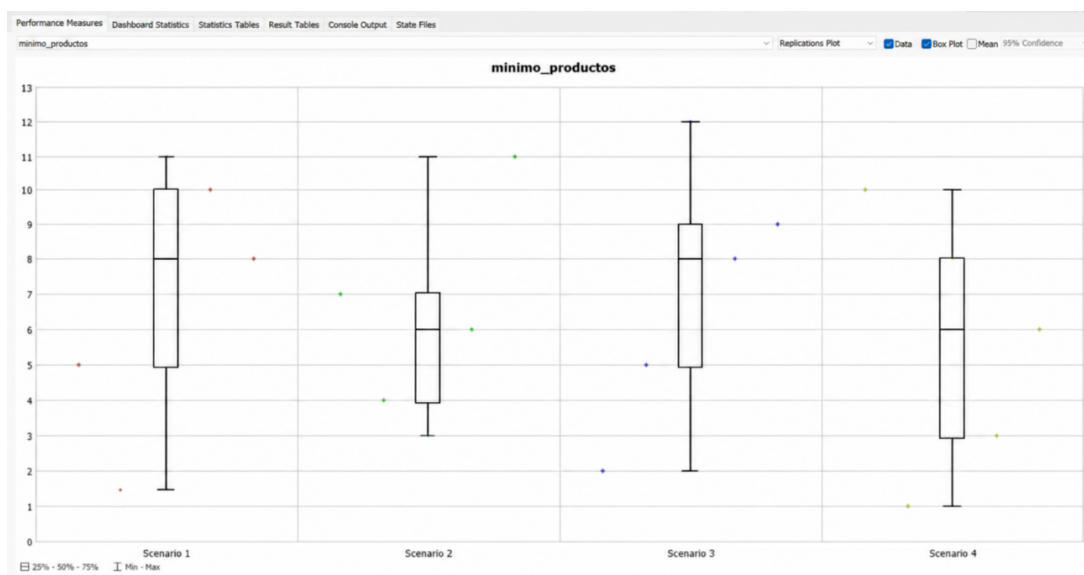


Figura 4.3: Resultados de `minimo_productos` para los escenarios de recarga en seco.

En el Escenario 1, la mediana se sitúa aproximadamente en 8 unidades. La mayor parte de los resultados se encuentra entre 5 y 10 unidades, con valores mínimos cercanos a 1-2 unidades y máximos alrededor de 11. Esto indica que la política baja mantiene, en general, el inventario en niveles aceptables, aunque en algunas réplicas algún producto llega a estar muy cerca de agotarse. Es una política bastante segura, pero no elimina completamente el riesgo de alcanzar niveles bajos.

En el Escenario 2, la mediana baja hasta aproximadamente 6 unidades. Los resultados centrales se encuentran entre unas 4 y 7 unidades, con un mínimo alrededor de 3 y un



máximo cercano a 11. Esta política mantiene el inventario en valores positivos, pero deja menos margen de seguridad que el Escenario 1. Aunque no se producen roturas de stock, el sistema trabaja más cerca de niveles críticos.

En el Escenario 3, la mediana vuelve a situarse aproximadamente en 8 unidades. Los valores centrales se encuentran entre unas 5 y 9 unidades, con mínimos cercanos a 2 y máximos alrededor de 12. Este resultado es especialmente interesante, porque indica que una política más agrupada puede mantener un nivel de inventario similar al escenario más reactivo. Es decir, el sistema consigue esperar a una mayor agrupación de pallets sin que el stock mínimo se deteriore de forma significativa.

En el Escenario 4, la mediana se sitúa aproximadamente en 6 unidades. La dispersión es notable, con valores desde aproximadamente 1 hasta 10 unidades. Aunque no aparecen valores negativos, el escenario alto es el que muestra uno de los márgenes de seguridad más bajos. Al esperar más para solicitar el camión, algunos productos llegan a niveles reducidos antes de ser repuestos. Esto aumenta el riesgo operativo, especialmente si en una situación real se produjera un aumento puntual de demanda o un retraso en la llegada del camión.

La comparación de esta métrica permite concluir que todos los escenarios mantienen el inventario en valores positivos, por lo que ninguno provoca rotura directa de stock en las réplicas analizadas. Sin embargo, los escenarios 2 y 4 presentan medianas más bajas, mientras que los escenarios 1 y 3 mantienen un nivel mínimo de producto más elevado.

El análisis conjunto de `inputsCola` y `minimoProductos` permite evaluar el equilibrio entre eficiencia logística y seguridad de inventario.

El Escenario 1 es una política conservadora. Genera un volumen de recarga elevado y mantiene el stock mínimo en una mediana aceptable, cercana a 8 unidades. Sin embargo, al ser más reactivo, puede provocar más movimientos de recarga de los necesarios y un menor aprovechamiento de la capacidad del camión. Es una opción segura, pero no necesariamente la más eficiente. El Escenario 2 genera la mayor cantidad de entradas en la cola de recarga, con una mediana cercana a 69. Sin embargo, el stock mínimo baja hasta una mediana aproximada de 6 unidades. Esto indica que, aunque hay bastante actividad de reposición, la política no mantiene el inventario tan protegido como los escenarios 1 y 3. Puede considerarse una opción operativa, pero no destaca por seguridad de inventario.

El Escenario 3 presenta el mejor equilibrio global. Aunque la mediana de entradas en cola no es la más alta, se mantiene en un nivel razonable y permite una mayor agrupación de la recarga. Al mismo tiempo, el stock mínimo alcanza una mediana aproximada de 8 unidades, similar a la del escenario más conservador. Esto significa que se consigue un buen aprovechamiento del camión sin reducir de forma importante la seguridad del inventario.

El Escenario 4 reduce el número de entradas de recarga y permite una política más exigente en cuanto a agrupación. Sin embargo, el stock mínimo presenta una mediana más baja, cercana a 6 unidades, y valores mínimos próximos a 1. Aunque no se llega a stock negativo, esta configuración trabaja con menos margen de seguridad. Por tanto, es una política viable, pero más arriesgada.

A partir de los resultados obtenidos, **el Escenario 3 se considera la opción más adecuada** para la política de recarga de seco, con **7 pallets**.

Este escenario consigue un equilibrio favorable entre las dos métricas analizadas. Por un lado, mantiene una cantidad de entradas en cola razonable, lo que indica que la recarga se activa de forma suficiente para alimentar el inventario. Por otro lado, conserva un stock mínimo con una mediana aproximada de 8 unidades, similar al escenario más conservador y superior a los escenarios 2 y 4.

## 4.4. Experimentación 2: política de recarga en fresco

En esta experimentación se han realizado las mismas *performance measures* y se han mantenido los mismos parámetros de la prueba anterior, pero aplicados a la nave refrigerada. El interés aquí no está solo en comprobar la seguridad del inventario, sino también en ver cómo afecta la lógica de agrupación de pallets a un entorno más condicionado por la circulación interna y por la necesidad de atravesar la esclusa.

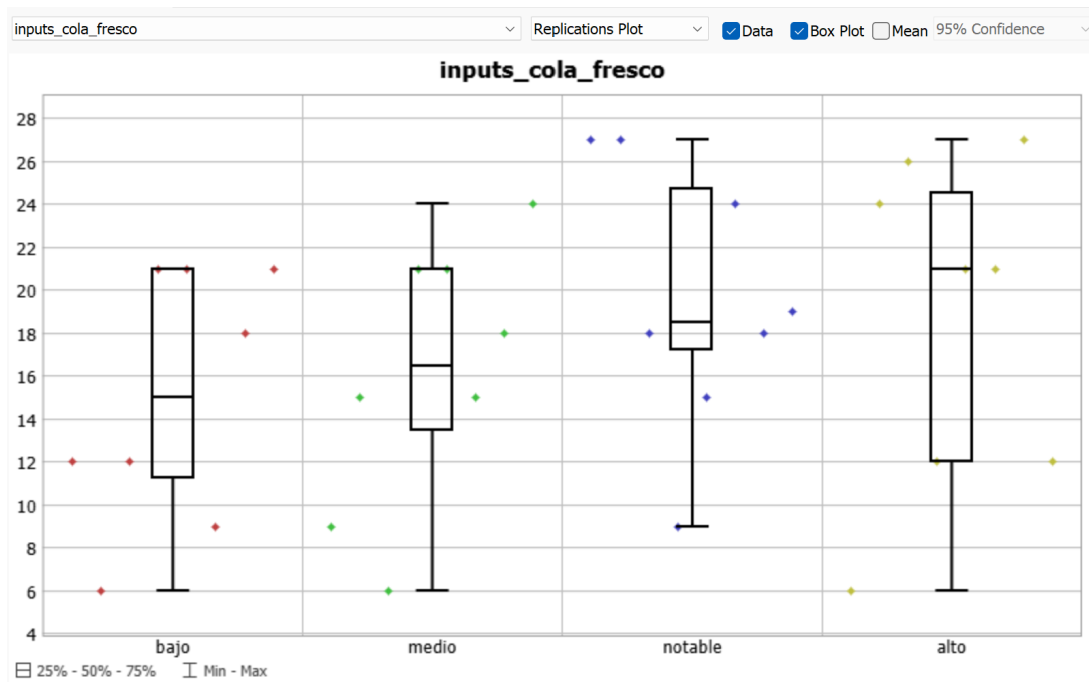


Figura 4.4: Resultados de entradas en cola de recarga para la sección de fresco.

El primer gráfico representa el número de entradas en la cola de recarga de fresco para cada uno de los escenarios. Esta métrica permite observar cómo afecta la política de agrupación de pallets al flujo de reposición que llega a la sección refrigerada. En el escenario bajo, la mediana se sitúa aproximadamente alrededor de 15 entradas. Los valores presentan una dispersión moderada, con resultados que van aproximadamente desde 6 hasta 21 entradas. Esto indica que la política más reactiva genera entradas de recarga, pero no necesariamente consigue un volumen elevado.

En el escenario medio, la mediana aumenta ligeramente hasta aproximadamente 16 o 17 entradas. La dispersión es parecida a la del escenario bajo, aunque se alcanzan valores algo superiores, cercanos a 24 entradas. Agrupar algo más los productos antes de solicitar el camión permite aumentar ligeramente el flujo sin cambiar de forma drástica el comportamiento del sistema.

En el escenario notable, se observa una mejora clara. La mediana se sitúa aproximadamente en torno a 18 o 19 entradas, con valores centrales bastante altos. Esto indica que esta política permite un mejor aprovechamiento de la recarga, generando un flujo más elevado hacia la cola de fresco.

En el escenario alto, la mediana es incluso algo superior, situándose aproximadamente alrededor de 21 entradas. Sin embargo, la dispersión es mayor, con valores que van desde aproximadamente 6 hasta 27 entradas. Por tanto, aunque el escenario alto puede aprovechar bien el camión cuando se activa, su comportamiento es menos estable.

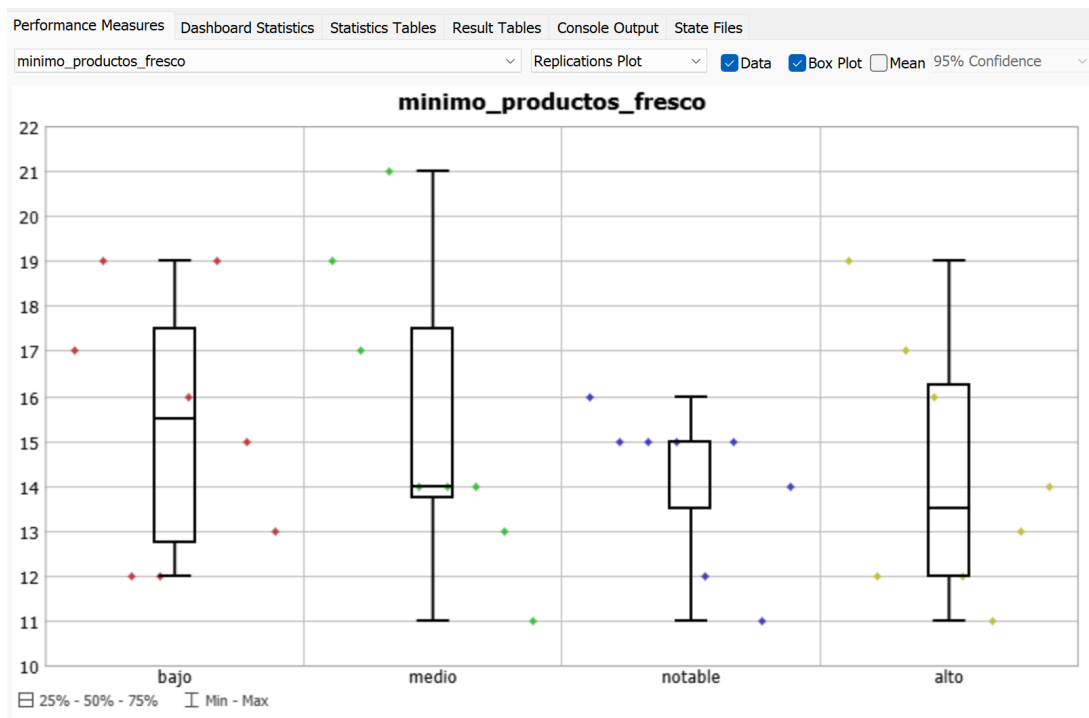


Figura 4.5: Resultados de stock mínimo alcanzado en la sección de fresco.

El segundo gráfico representa el mínimo nivel de stock alcanzado por cualquier producto de la sección de fresco. Esta es la métrica más importante para evaluar si la política de recarga es segura.

En el escenario bajo, la mediana del stock mínimo se sitúa aproximadamente alrededor de 15 o 16 unidades. Los valores se mantienen entre unas 12 y 19 unidades. Esto indica que la política reactiva mantiene el inventario en niveles seguros.

En el escenario medio, la mediana baja ligeramente, situándose aproximadamente alrededor de 14 unidades. El rango de resultados se extiende entre 11 y 21 unidades. Aunque sigue siendo una política segura, aparece una mayor variabilidad.

En el escenario notable, el stock mínimo presenta un comportamiento especialmente interesante. La mediana se sitúa aproximadamente en torno a 15 unidades y la dispersión es más reducida que en otros escenarios. La mayoría de los resultados se concentran entre unas 13,5 y 15 unidades, con valores mínimos cercanos a 11 y máximos alrededor de 16.

En el escenario alto, la mediana desciende hasta aproximadamente 13 o 14 unidades. Aunque no aparecen valores negativos ni roturas de stock, el inventario mínimo es menor que en los escenarios bajo y notable.

Por tanto, desde el punto de vista de seguridad de inventario, todos los escenarios son aceptables porque no se observan valores negativos. Sin embargo, no todos ofrecen el mismo equilibrio. El escenario notable genera un flujo de recarga alto y, al mismo tiempo, mantiene el nivel mínimo de productos en valores seguros y bastante estables. Esta combinación lo convierte en la opción más adecuada para la sección de fresco.

En consecuencia, también para fresco **la mejor opción es el parámetro notable, equivalente al escenario de 7 pallets**. No es una elección extrema, pero sí la más equilibrada entre aprovechamiento del camión, estabilidad del inventario y seguridad de funcionamiento.

## 4.5. Experimentación 3: número de combiners en seco

El objetivo de esta experimentación es evaluar cómo el número de combiners afecta la capacidad de paletizado en la sección de seco. Se busca determinar la configuración óptima que maximice la producción de pallets sin incrementar innecesariamente los recursos ni generar cuellos de botella.

Se definieron tres escenarios en función del número de combiners disponibles: Cada escenario se implementó manteniendo constantes el resto de parámetros de la planta, de modo que las diferencias observadas en producción y tiempos se pudieran atribuir únicamente al número de combiners.

| Escenario | Opción | Valor |
|-----------|--------|-------|
| 1         | Mínimo | 2     |
| 2         | Medio  | 3     |
| 3         | Máximo | 4     |

Figura 4.6: Escenarios de paletizado en seco.

Para cada escenario se analizaron los siguientes *performance measures*: Para cada escenario se analizaron los siguientes *performance measures*:

- **pallets\_completados**: representa el número total de pallets completados en los combiners de la sección de seco. Esta métrica refleja directamente la capacidad productiva de la zona de paletizado.
- **tiempo\_medio\_pallet**: tiempo medio que tarda un pallet en completarse. Esta métrica indica la eficiencia operativa de cada combiner individual y permite evaluar si la adición de combiners influye en la rapidez de montaje.

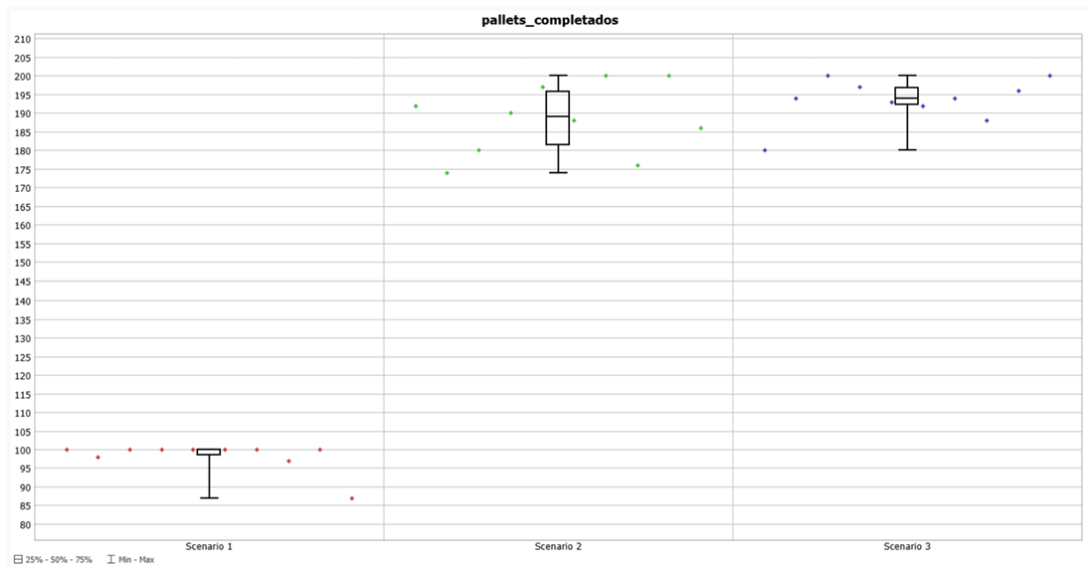


Figura 4.7: Pallets completados para los distintos escenarios de combiners en seco.

Escenario 1 (2 combiners): La mediana de pallets completados se sitúa alrededor de 98 pallets, con un rango que va desde 87 hasta 100. Esto indica que la producción está limitada, reflejando un cuello de botella en la sección de paletizado. La baja cantidad de combiners impide que todos los pedidos generados por los ASRS y transportados por los conveyors se procesen de manera eficiente.

Escenario 2 (3 combiners): La mediana aumenta significativamente hasta aproximadamente 188-190 pallets, con valores que oscilan entre 174 y 200. Este incremento demuestra que

la adición de un tercer combiner elimina gran parte del cuello de botella, permitiendo que la producción alcance niveles casi máximos y más uniformes entre réplicas. La dispersión de los resultados también disminuye, mostrando una mayor estabilidad del sistema.

Escenario 3 (4 combiners): La mediana se sitúa alrededor de 194-196 pallets, con un aumento marginal respecto al Escenario 2. Aunque se observa un ligero incremento en el máximo alcanzado, la mejora no es significativa. Esto sugiere que, a partir de tres combiners, la limitación ya no está en la capacidad de paletizado sino en otros factores del sistema, como la llegada de productos desde los ASRS o la velocidad de los AGVs.

Como se puede observar, el aumento de combiners de 2 a 3 produce un incremento sustancial de pallets completados, mientras que añadir un cuarto combiner aporta un beneficio marginal. Por tanto, 3 combiners representan un equilibrio óptimo entre capacidad y eficiencia. Pero para analizar de forma completa la opción óptima, antes debemos medir el tiempo medio por pallet para evaluar la eficiencia operativa de los combiners:

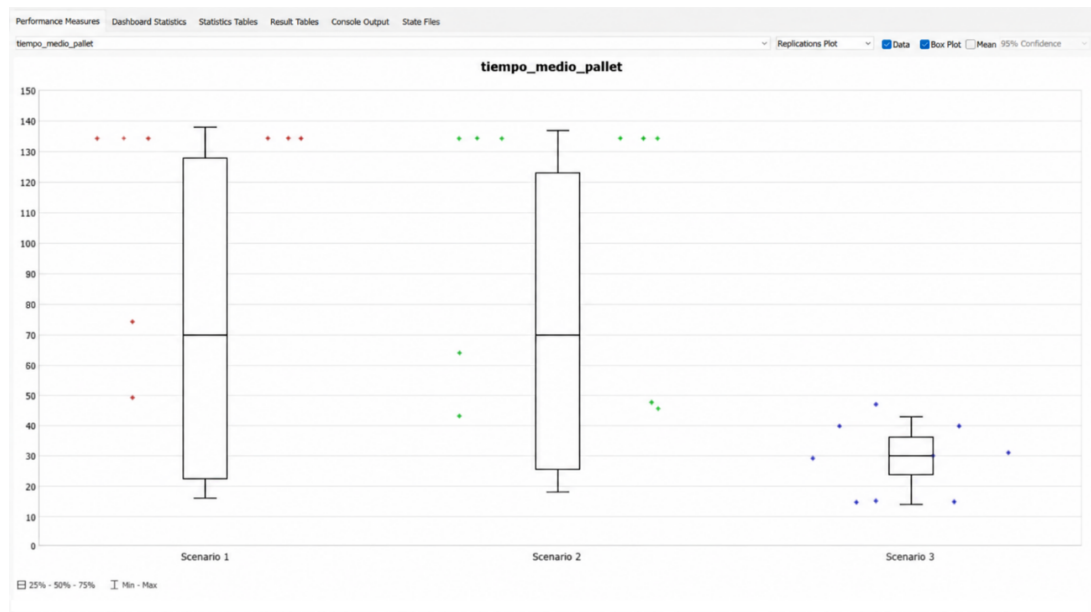


Figura 4.8: Tiempo medio de paletizado en la sección de seco.

Escenario 1 (2 combiners): La mediana del tiempo medio se sitúa aproximadamente entre 70 y 130 unidades de tiempo, con una dispersión elevada. Esto refleja que, con solo 2 combiners, los pallets tardan más en completarse debido a la saturación de la capacidad disponible.

Escenario 2 (3 combiners): La mediana se mantiene estable, con valores similares a los del Escenario 1, aunque ligeramente inferiores en algunas réplicas. Esto indica que, al eliminar el cuello de botella, la eficiencia de cada combiner individual mejora o se mantiene sin degradarse.

Escenario 3 (4 combiners): Los valores de tiempo medio se mantienen similares a los

del Escenario 2. Esto confirma que añadir un cuarto combiner no influye de manera significativa en el tiempo de montaje por pallet.

Por tanto, el tiempo medio de paletizado no se ve penalizado al aumentar el número de combiners y permanece estable entre los escenarios 2 y 3. Esto refuerza la idea de que tres combiners son suficientes para procesar la mayoría de los pallets de forma eficiente.

El análisis de la experimentación indica que **el número óptimo de combiners en la sección de seco es 3 (Escenario 2 – Medio)**. Esta configuración:

- Permite maximizar la producción de pallets completados.
- Mantiene el tiempo medio de paletizado estable, asegurando eficiencia operativa.
- Evita sobredimensionar recursos, ya que un cuarto combiner aporta beneficios marginales.

## 4.6. Experimentación 4: AGVs y combiners en fresco

La zona de fresco requiere un análisis conjunto, ya que aquí los productos no llegan directamente a los combiners, sino que son transportados por AGVs desde las zonas de salida de los ASRS y deben atravesar la esclusa. Por tanto, el rendimiento de esta zona no depende únicamente del número de combiners ni únicamente del número de AGVs. Si existen pocos AGVs, los productos pueden acumularse en las colas de origen y no alimentar suficientemente los combiners. Si existen demasiados AGVs, la red puede congestionarse, especialmente en zonas críticas como la esclusa. Del mismo modo, aumentar el número de combiners no garantiza una mejora si no existe suficiente flujo de productos para alimentarlos. Los parámetros utilizados en la experimentación fueron los siguientes:

Tabla 4.1: Parámetros utilizados en la experimentación de fresco.

| Parámetro            | Opción | Valor |
|----------------------|--------|-------|
| num_combiners_fresco | Unico  | 1     |
| num_combiners_fresco | Dual   | 2     |
| num_agvs_fresco      | Seis   | 6     |
| num_agvs_fresco      | Siete  | 7     |
| num_agvs_fresco      | Ocho   | 8     |

Las combinaciones analizadas fueron:

Tabla 4.2: Combinaciones experimentales utilizadas en la sección de fresco.

| Escenario   | num_agvs_fresco | num_combiners_fresco |
|-------------|-----------------|----------------------|
| seis+unico  | seis            | único                |
| seis+dual   | seis            | dual                 |
| siete+unico | siete           | único                |
| siete+dual  | siete           | dual                 |
| ocho+unico  | ocho            | único                |
| ocho+dual   | ocho            | dual                 |

Las medidas de rendimiento utilizadas fueron:

- `pallets_completados_fresco`: número de pallets completados en la zona de fresco;
- `tiempo_medio_pallet_fresco`: tiempo medio de montaje en los combiners de fresco.

El análisis del gráfico muestra claramente cómo interactúan el número de AGVs y la cantidad de combiners:

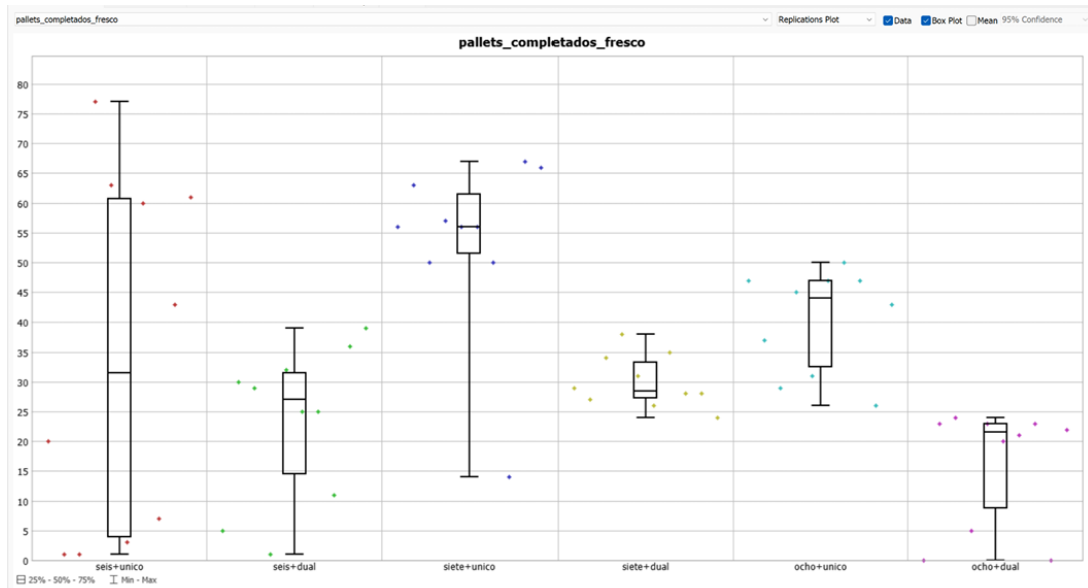


Figura 4.9: Pallets completados en la experimentación conjunta de AGVs y combiners en fresco.

- **Seis+único (6 AGVs, 1 combiner)**: Mediana de pallets completados alrededor de 31, con valores que van desde 1 hasta 77. La producción es limitada y presenta alta dispersión, reflejando que los 6 AGVs apenas consiguen alimentar de manera constante un único combiner.
- **Seis+dual (6 AGVs, 2 combiners)**: Mediana ligeramente menor, alrededor de 28-30 pallets, con dispersión similar. Añadir un combiner extra sin incrementar los AGVs



no mejora la producción y genera variabilidad adicional, ya que los AGVs no pueden abastecer eficientemente ambas líneas.

- **Siete+único (7 AGVs, 1 combiner):** Mediana de pallets completados aumenta significativamente a 55-57 pallets. Los AGVs adicionales permiten mantener un flujo constante de productos hacia el combiner, reduciendo la dispersión y mejorando la estabilidad de la producción.
- **Siete+dual (7 AGVs, 2 combiners):** Mediana cae a 29 pallets aproximadamente, con dispersión notable. Aunque hay más combiners, los AGVs no pueden alimentar de forma eficiente ambas líneas, generando retrasos y menor producción.
- **Ocho+único (8 AGVs, 1 combiner):** Mediana alrededor de 44-45 pallets, con ligera dispersión. El octavo AGV no aporta mejoras sustanciales respecto a siete AGVs, indicando que el cuello de botella sigue siendo el flujo coordinado hacia el único combiner.
- **Ocho+dual (8 AGVs, 2 combiners):** Mediana de 22-23 pallets y alta dispersión. La combinación de AGVs excesivos y combiners duales provoca congestión y coordinación ineficiente, reduciendo la producción.

El tiempo medio de paletizado en la sección de fresco mide cuánto tarda un pallet en completarse desde que se inicia su montaje hasta que se termina en el combiner. Esta métrica es crucial porque refleja no solo la eficiencia de los combiners, sino también la coordinación del flujo de productos transportados por los AGVs a través de la esclusa. A diferencia de la sección de seco, en fresco los pallets dependen de la llegada continua de productos desde los ASRS mediante AGVs, por lo que el tiempo de montaje puede verse afectado tanto por la capacidad del combiner como por la congestión en la red de transporte.

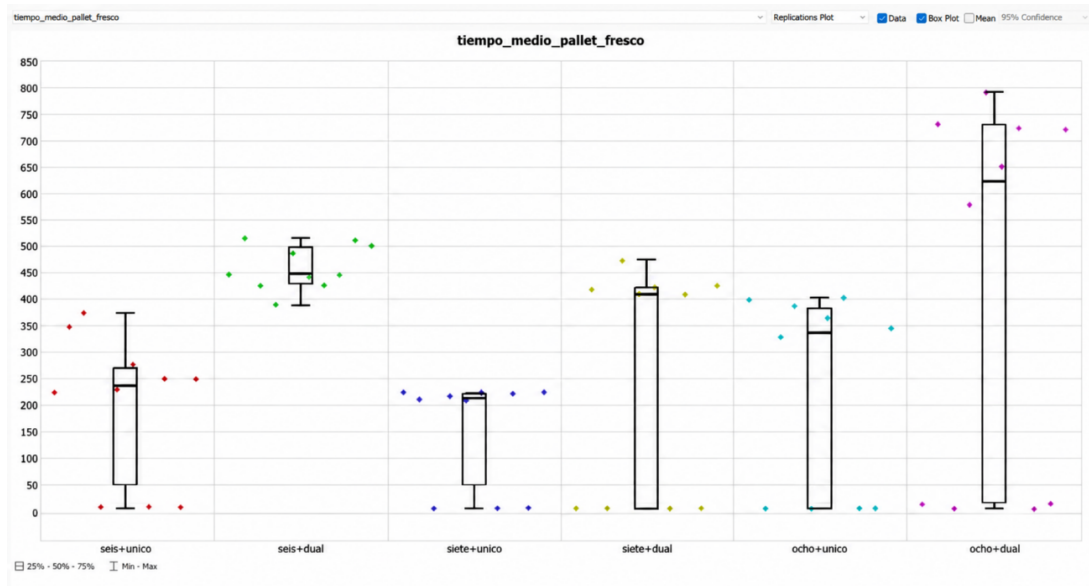


Figura 4.10: Tiempo medio de paletizado para las configuraciones analizadas en fresco.

- **Escenario seis+único (6 AGVs, 2 combiners):** La mediana de pallets completados se sitúa alrededor de 31, con valores que oscilan entre 1 y 77. Esto indica que, aunque la producción puede alcanzar cifras altas en algunos casos, la combinación de pocos AGVs y solo 2 combiners genera variabilidad y limita la capacidad productiva en general.
- **Escenario seis+dual (6 AGVs, 3 combiners):** La mediana aumenta a aproximadamente 28-30 pallets, pero la dispersión es todavía elevada. La adición de un combiner extra mejora la capacidad del sistema, pero el número limitado de AGVs impide aprovecharlo de manera consistente.
- **Escenario siete+único (7 AGVs, 2 combiners):** La mediana sube significativamente a alrededor de 56 pallets. Esto demuestra que aumentar los AGVs, incluso manteniendo solo 2 combiners, mejora notablemente la alimentación de productos a los combiners y la producción general.
- **Escenario siete+dual (7 AGVs, 3 combiners):** Se observa una ligera reducción en la dispersión de resultados y una mediana cercana a 29 pallets. Aunque hay más combiners, la combinación no logra superar la eficiencia del escenario siete+único, indicando que la coordinación entre flujo de AGVs y combiners es crítica.
- **Escenario ocho+único (8 AGVs, 2 combiners):** La mediana se mantiene alrededor de 44-45 pallets. Esto refleja que añadir un octavo AGV no produce un aumento proporcional de producción; el cuello de botella empieza a depender de los combiners y la esclusa.
- **Escenario ocho+dual (8 AGVs, 3 combiners):** La mediana baja a aproximadamente 22-23 pallets, y los valores extremos muestran alta dispersión. Esto indica que la

combinación de exceso de AGVs con más combiners provoca congestión y coordinación ineficiente, reduciendo la producción.

Entonces, **el escenario siete+único (7 AGVs y un solo combiner) es claramente el más eficiente**, combinando alta producción con estabilidad. El análisis muestra que una única línea de combiner permite aprovechar mejor el flujo de AGVs sin generar cuellos de botella ni complejidad innecesaria. Además, mantiene tiempos medios consistentes y bajos, confirmando que concentrar la paletización en una sola línea maximiza la eficiencia y evita retrasos. Los escenarios duales o con exceso de AGVs muestran mayor dispersión y tiempos extremos, lo que reduce la eficiencia general.

## 4.7. Validación de resultados

Una vez realizados los experimentos anteriores, se procede a validar el comportamiento global de la solución propuesta a partir de los KPIs definidos inicialmente: pallets completados, tiempo medio de formación de pallet, stock mínimo positivo y aprovechamiento de los camiones.

- **Stock mínimo positivo y aprovechamiento de camiones:** tanto para seco como para fresco corresponde al escenario notable, equivalente a solicitar la recarga con 7 pallets. Esta configuración representa un punto intermedio entre una política demasiado reactiva, que provocaría demasiados movimientos de reposición, y una política demasiado exigente, que retrasaría en exceso la llegada del camión y aumentaría el riesgo de trabajar con inventarios bajos. En seco, el escenario de 7 pallets mantiene el stock mínimo en torno a 8 unidades, al mismo tiempo que reduce la necesidad de recargas excesivamente frecuentes. En fresco, el mismo criterio también resulta adecuado, ya que mantiene el stock mínimo en valores positivos y relativamente estables, con medianas próximas a 15 unidades. Por tanto, desde el punto de vista del KPI de stock mínimo positivo, la solución queda validada: en ningún escenario seleccionado se observan roturas de stock y el sistema conserva margen suficiente para absorber variaciones normales de demanda.
- **Pallets completados:** los resultados de la experimentación de combiners en seco muestran que la configuración óptima es la de 3 combiners, la opción más eficiente para seco: aumenta claramente la producción respecto al escenario limitado y evita añadir recursos que apenas mejoran el rendimiento. Además, para medir la cantidad de pallets expedidos por unidad de tiempo, se ha realizado en un Dashboard un análisis de la cantidad de outputs por hora, obteniendo el siguiente resultado:

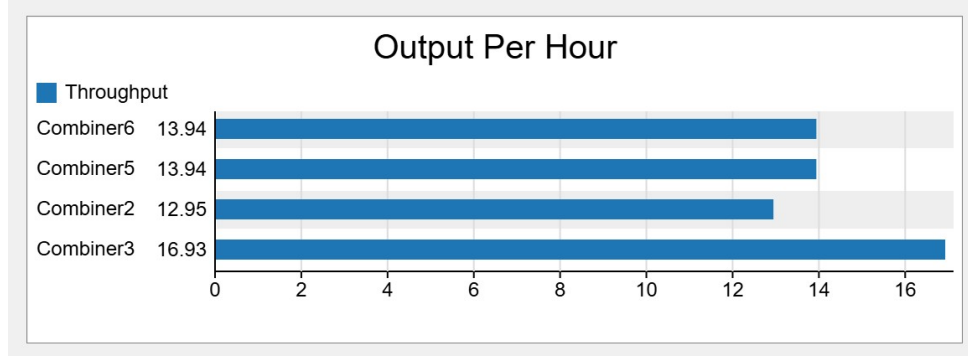


Figura 4.11: Pallets por cola expedidos por hora.

Como se puede observar, se realizan unos 58 pallets en tan solo una hora. Según los datos considerados, un trabajador es capaz de montar aproximadamente 21 pallets durante una jornada de 7 horas, lo que equivale a una productividad manual media de 3 pallets por hora. Comparando ambos valores:

$$n_{trabajadores} = pallets_{automatizacion} \div \left( \frac{pallets_{operario}}{t_{horas}} \right) = 58 \div \left( \frac{21}{7} \right) = 19,33 \quad (4.1)$$

Obtenemos un resultado muy positivo, ya que la planta automatizada produce por hora lo mismo que unos 19 trabajadores de forma manual. En una jornada de 7 horas, la planta haría 406 pallets frente a los 21 pallets por trabajador. Dicho de otro modo, para igualar la producción horaria de la planta sería necesario disponer de unos 19 o 20 operarios dedicados exclusivamente al montaje manual de pallets. Por tanto, el nivel de optimización alcanzado puede considerarse muy elevado. La automatización permite pasar de una productividad manual de 3 pallets/h por trabajador a una producción global de 58 pallets/h, manteniendo un ritmo continuo y reduciendo la necesidad de intervención humana directa en el proceso de paletizado.

- **Tiempo medio de formación de pallet:** Según la gráfica de tiempo medio de paletizado en seco, para la configuración seleccionada de 3 combiners —Escenario 2— la mediana se sitúa aproximadamente en 70 segundos.

En la sección de fresco, la configuración seleccionada era 7 AGVs y 1 combiner. En la gráfica de tiempo medio de paletizado en fresco, esa configuración presenta una mediana aproximada de 210-220 segundos. Aunque el tiempo medio de fresco es mayor que en seco, es lógico porque el pallet depende de la llegada de productos mediante AGVs, de la coordinación en la red y del paso por la esclusa, no solo del combiner.

Estos resultados nos muestran que cada pallet tarda solamente entre 70 y 220 segundos en montarse. Este intervalo demuestra que el proceso de paletizado se realiza en tiempos reducidos y compatibles con una operación logística de alta capacidad.

- **Cuellos de botella:** los cuellos de botella en la planta se pueden comprobar fácilmente observando el contenido medio de las colas durante la simulación. Para comprobarlo, se ha realizado un gráfico de barras comprobando el Average Content de cada cola que representa un punto crítico en el almacén:

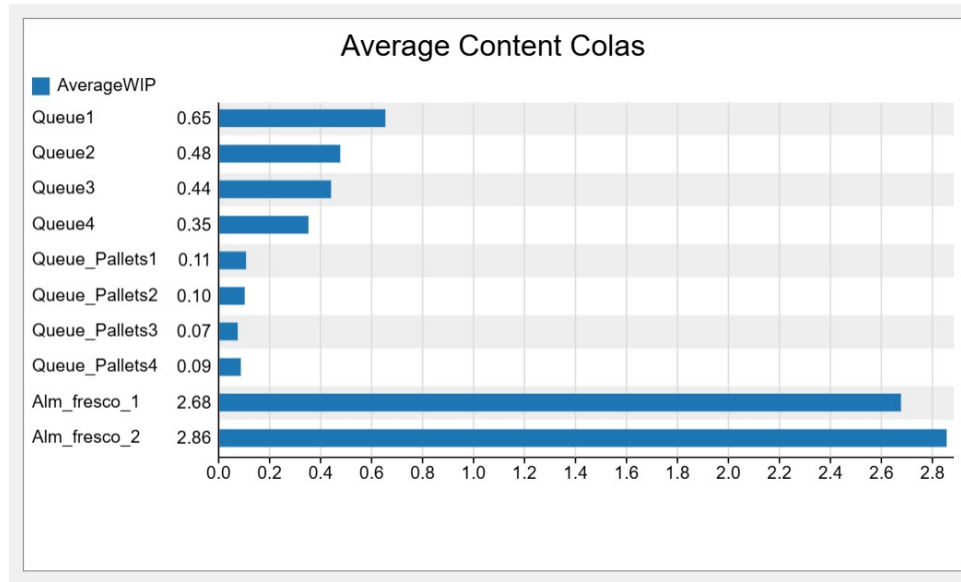


Figura 4.12: Contenido medio de las colas de la nave.

Los valores obtenidos indican que la expedición no depende únicamente de una única cola saturada, sino que el flujo se reparte entre varios puntos de salida. Aunque las colas del amacén refrigerado presenten una producción algo superior, el contenido promedio es muy bajo, lo que confirma que la planta es capaz de mantener una salida continua de pallets y que los recursos de expedición trabajan con un nivel de actividad significativo.

En conclusión, los resultados experimentales y las figuras de validación muestran que la planta automatizada funciona de manera coherente con los objetivos definidos. La solución no se limita a aumentar recursos de forma indiscriminada, sino que selecciona configuraciones equilibradas para cada zona: 7 pallets como política de recarga en seco y fresco, 3 combiners en seco y 7 AGVs con 1 combiner en fresco. Esta combinación permite mantener la producción, evitar roturas de stock, controlar los tiempos de paletizado y reducir acumulaciones internas. Por tanto, el modelo queda validado como una propuesta técnicamente viable y operativamente equilibrada para la automatización de la nave logística.

# Aspectos Avanzados de Control y Navegación

## 5.1. Sensorización e Integración

Para trasladar el modelo de FlexSim a una planta realista, se ha definido una estrategia de sensorización que cubre los elementos críticos del sistema: AGVs, operarios, estaciones de picking, combiners, colas de consolidación y zonas de carga y descarga. El objetivo no es solo detectar presencia, sino dar sentido operativo a cada señal para que el modelo pueda decidir, parar, avanzar o esperar con criterios coherentes con una implantación industrial.

### 5.1.1. Sensorización

La sensorización se ha planteado como una capa de detección distribuida. En una planta real, esa capa se materializaría con sensores de proximidad, lectores de identificación, barreras fotoeléctricas, cámaras y sistemas de localización. En la simulación, la misma idea se representa mediante agentes de proximidad, Control Areas, eventos sobre colas y etiquetas que acompañan a cada ítem o recurso.

No basta con detectar que hay un elemento en una posición concreta. Cada sensor debe tener una función clara dentro del proceso: evitar colisiones, validar la llegada de material, comprobar que una orden está completa o impedir que un recurso entre en una zona ocupada. Esa es la razón por la que la sensorización se ha diseñado pensando en la operativa y no solo en la geometría del modelo.

- **AGVs:** los AGVs se equiparían con sensores de proximidad y navegación realistas. En una implantación física, lo normal sería disponer de LIDAR 2D o 3D para detectar objetos, otros AGVs y operarios en un radio amplio, sensores ultrasónicos o de infrarrojos para la detección inmediata de proximidad, encoders y odometría para medir velocidad y posición, y sensores RFID para reconocer localizaciones críticas como zonas de carga, estanterías o colas de consolidación. En el modelo, esa capacidad se aproxima mediante agentes de cruce, zonas de exclusión y reglas de parada que impiden que dos vehículos compartan un punto de conflicto.

Esta sensorización es la base del comportamiento de seguridad. Si el AGV detecta un obstáculo, otro vehículo o un operario en su radio de seguridad, el sistema interpreta la señal como una ocupación y aplica una detención temporal o una reducción de velocidad. Cuando la zona queda libre, el recurso retoma la marcha. Con ello se reproduce una navegación segura y cercana a la de un sistema real. Por tanto:

- **LIDAR 2D/3D:** proporciona un mapa en tiempo real del entorno, detectando obstáculos estáticos y dinámicos. Esto permite al AGV anticipar colisiones y ajustar su trayectoria de manera precisa.
- **Sensores ultrasonidos e infrarrojos:** instalados alrededor del vehículo para detección inmediata de objetos cercanos y activación de parada de emergencia.
- **Encoders y odometría:** permiten medir la velocidad y posición con alta precisión, asegurando que los cálculos de distancia para la parada sean fiables.
- **RFID:** detecta puntos de referencia en la planta como zonas de picking, carga o colas de consolidación, asegurando que el AGV reconozca su posición relativa y el destino correcto del pedido.

Estos sensores se configuran para reaccionar automáticamente cuando otro AGV o un operario se encuentra dentro del radio de seguridad, deteniendo el AGV temporalmente y reanudando su movimiento solo cuando la trayectoria está libre.

- **Estaciones de picking y zonas de carga/descarga:** las estaciones de picking, las zonas de carga y las zonas de descarga deben disponer de sensores que confirmen la llegada y salida de pallets o items. En una planta industrial esto se conseguiría con sensores de peso, fotocélulas, cámaras de visión artificial y sensores de presión o interruptores mecánicos situados en las mesas o tolvas.

En la simulación, esa lógica se traduce en eventos On Entry y On Exit, en colas con capacidad limitada y en el uso de etiquetas para verificar que el item transportado coincide con la orden asignada. De esta manera, la zona no actúa como un simple punto de paso, sino como un control de validación que confirma que el material correcto entra, sale y avanza al siguiente bloque del proceso. Por tanto:

- **Fotocélulas y sensores de peso:** detectan la llegada y salida de items o pallets, permitiendo contabilizar cada producto que entra o sale de la estación.
- **Cámaras con visión artificial:** identifican códigos de producto y etiquetas para verificar que los items coinciden con la orden asignada.
- **Sensores de presión o interruptores mecánicos:** aseguran la correcta colocación de items en bandejas o mesas de picking, evitando errores de manipulación.

- **Combiners y colas de consolidación:** los combiners solo deben cerrar un pallet cuando la orden está completa y los items pertenecen a la misma referencia lógica. Para conseguirlo, en una planta real se emplearían sensores de presencia inductivos o capacitivos, lectores RFID o de código de barras y cámaras de verificación que comprueben el estado de cada puerto.

En el modelo, esta función se resuelve mediante labels, tablas de control y comprobaciones del OrderID. El sensor no solo detecta si hay producto, también valida si pertenece a la orden correcta y si ya han llegado todas las unidades necesarias. Si falta material, el pallet permanece en espera; si la orden está completa, el combiner autoriza el cierre. Esto evita mezclar pedidos y da coherencia al proceso de consolidación. Por tanto:

- **Sensores capacitivos o inductivos:** detectan la presencia de items en los puertos de los combiners y aseguran que solo se active la consolidación cuando se recibe el producto correcto.
  - **Lectores RFID o códigos de barras:** garantizan que solo los items de una misma orden se agrupan en el mismo pallet, evitando mezcla de pedidos.
  - **Cámaras de verificación:** supervisan la formación del pallet y la correcta posición de los productos.
- **Operarios y áreas de mantenimiento:** los operarios también deben estar sensorizados, porque en una planta automatizada comparten espacio con vehículos móviles. En una implantación real se podrían usar dispositivos wearables con UWB o BLE para localizar la posición de cada operario, además de cámaras de seguimiento de flujo para controlar densidad de personal y recorridos en tiempo real.

En FlexSim, esta misma idea se reproduce con el agente supervisor y con las zonas de seguridad alrededor de las rutas de AGVs. Cuando el operario entra en un área compartida, el sistema interpreta la señal como una ocupación sensible y obliga a los vehículos a detenerse o a cambiar su comportamiento. De esta forma se evita la coexistencia insegura entre personas y equipos automáticos. Por tanto:

- **Sensores de proximidad wearable (UWB/BLE):** permiten conocer la posición de cada operario y ajustar la velocidad y trayectoria para evitar colisiones con AGVs o con otros operarios.
  - **Cámaras de seguimiento de flujo:** monitorizan la densidad de operarios en pasillos y zonas de trabajo, ayudando a mantener la eficiencia y seguridad.
- **Áreas generales de la planta:** además de los puntos anteriores, la planta debe disponer de sensorización global en pasillos, zonas de tráfico y áreas ambientales. En una instalación real, esto incluiría sensores de flujo, cámaras de vigilancia, sensores de tem-



peratura y humedad en la zona de fresco, y sensores de apertura y cierre en puertas automáticas o accesos a las zonas de picking.

En la simulación, estas variables se representan como estados de ocupación, restricciones de acceso y condiciones que alteran la velocidad o la disponibilidad de los recursos. Su papel no es tan visible como el de un sensor de proximidad, pero resulta importante para que el modelo tenga sentido y responda de forma razonable ante cambios de contexto. Por tanto:

- **Sensores de flujo o cámaras de vigilancia:** instalados en pasillos y áreas de cruce para medir ocupación y densidad de tráfico de AGVs y operarios.
- **Sensores ambientales** (temperatura, humedad): críticos en la zona de fresco para asegurar condiciones de almacenamiento adecuadas.
- **Sensores de apertura/cierre:** puertas automáticas o barreras que registran entradas y salidas en zonas restringidas o de picking.

La selección de estos elementos responde a una idea sencilla: cada parte de la planta debe poder percibir lo que ocurre a su alrededor sin depender de una supervisión central continua. Eso hace que el modelo sea más realista, más escalable y también más fácil de adaptar si cambian los recursos o la distribución de la planta.

### 5.1.2. Integración

Para integrar todos los sensores, se propone un modelo de comunicación distribuido y jerárquico

- **Red de AGVs y operarios:** Cada AGV y operario actúa como un nodo inteligente, comunicando su posición, velocidad, estado de movimiento y detección de obstáculos mediante **Ethernet industrial (Ethernet/IP)**. Los nodos envían mensajes periódicos de broadcast que incluyen su ID, **OrderID** y etiquetas de sensores (**obstacleDetected**, **proximityAlert**). Los AGVs reciben esta información de los vecinos y ajustan su comportamiento en tiempo real, aplicando reglas de parada y reanudación.
- **Zonas de Picking, Combiners y Colas:** Los sensores de fotocélulas, RFID y cámaras se conectan a **controladores locales (PLC o microcontroladores)** que comunican datos al sistema central mediante **OPC UA**. Cada controlador transmite el estado de las colas, combiners y pallets, incluyendo qué items han sido recibidos y si el pallet está completo. Esta comunicación permite que los AGVs y el sistema central conozcan la disponibilidad de slots y puedan coordinar la entrega de items de manera precisa.

- **Integración centralizada:** En FlexSim, esta comunicación se traduce en labels, eventos y reglas de decisión. Cada sensor físico se representa como un agente virtual que dispara eventos `onProximity`, `onItemDetected` o `onEnter`. Los labels (`obstacleDetected`, `palletReady`, `OrderID`) reflejan el estado de cada sensor y se utilizan en las reglas de parada, reanudación y consolidación de pedidos.

## 5.2. Cruce operativo entre AGVs

Con el objetivo de trasladar el comportamiento observado en el sistema físico a la simulación desarrollada en FlexSim, se utilizaron los tiempos experimentales obtenidos en el proyecto de gestión de cruce para robots siguelíneas basado en ESP32-S3 y MQTT.

En dicho proyecto se realizaron varias pruebas reales con dos robots compartiendo un cruce común. Los tiempos de ocupación del cruce se obtuvieron manualmente a partir del vídeo de demostración, midiendo el intervalo comprendido entre el instante en que un robot detectaba el marcador de entrada del cruce y el instante en que abandonaba completamente la zona de conflicto. A partir de estas mediciones se obtuvo la siguiente tabla experimental.

Tabla 5.1: Tabla experimental de tiempos de cruce obtenidos en el sistema físico.

| Robot   | Inicio (s) | Fin (s) | Tiempo total (s) |
|---------|------------|---------|------------------|
| Robot 1 | 3,39       | 6,09    | 2,70             |
| Robot 2 | 3,59       | 7,26    | 3,67             |

En estas pruebas, el Robot 1 fue el primero en acceder al cruce, mientras que el Robot 2 tuvo que esperar hasta que el primero liberó la zona de conflicto. Por este motivo, el tiempo del Robot 2 es superior, ya que incluye tanto el tiempo de espera como el tiempo real de cruce.

Para la simulación en FlexSim se tomó como referencia el tiempo de ocupación completo del Robot 1, es decir, 2,70 s, ya que representa el tiempo necesario para atravesar y liberar completamente el cruce sin esperas intermedias. Sin embargo, los robots físicos utilizados en las pruebas circulaban aproximadamente a 0,4 m/s, mientras que los AGVs implementados en FlexSim se desplazan a 1,5 m/s. Debido a esta diferencia de velocidades, fue necesario escalar los tiempos experimentales para obtener un valor equivalente en la simulación. Asumiendo que la longitud efectiva del cruce es la misma en ambos casos, se utilizó una proporcionalidad inversa entre velocidad y tiempo:

$$t_{AGV} = t_{robot} \cdot \left( \frac{v_{robot}}{v_{AGV}} \right) \quad (5.1)$$

Sustituyendo los valores experimentales:

$$t_{AGV} = 2,70 \cdot \left( \frac{0,4}{1,5} \right) = 0,72 \text{ exts} \quad (5.2)$$

Por tanto, el tiempo de espera configurado para el segundo AGV en FlexSim fue aproximadamente de 0,7 s, utilizando finalmente un valor práctico de 0,75 s para introducir un pequeño margen de seguridad frente a posibles desfases debidos a aceleraciones, tiempos de reacción o discretización de eventos en la simulación.

Gracias a este ajuste, el comportamiento de los AGVs en FlexSim reproduce de forma aproximada el funcionamiento observado en el sistema físico real, garantizando que un vehículo pueda atravesar el cruce mientras el segundo permanece detenido el tiempo suficiente para evitar conflictos o colisiones. En la siguiente imagen se puede observar el comportamiento del sistema físico, en el que un robot se detiene mientras el otro cruza.

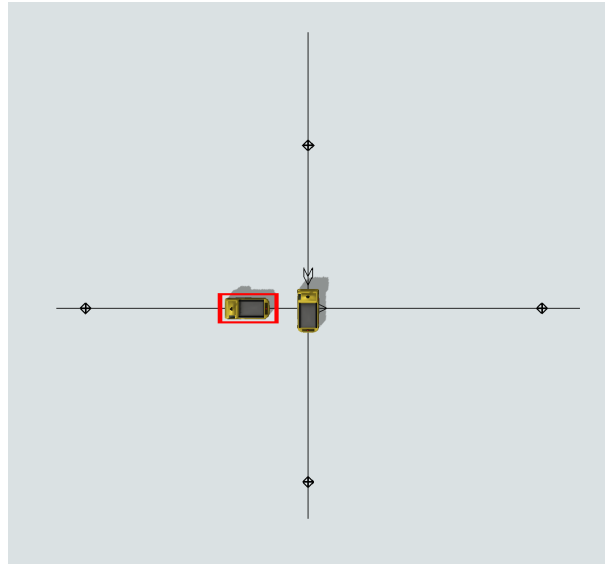


Figura 5.1: Comportamiento del cruce en el sistema físico de referencia.

Aplicado a nuestro caso concreto, el cruce se podría producir en diferentes tramos del AGV network. En estos cruces, gracias a la aplicación del agente de proximidad, se evitan las colisiones entre AGVs. Como se puede observar, los círculos señalan los puntos críticos donde los AGVs se encuentran. Además, para gestionar el paso de la puerta, al ser un camino crítico bidireccional en el que solo puede pasar un AGV a la vez, se gestiona también con un Control Area.

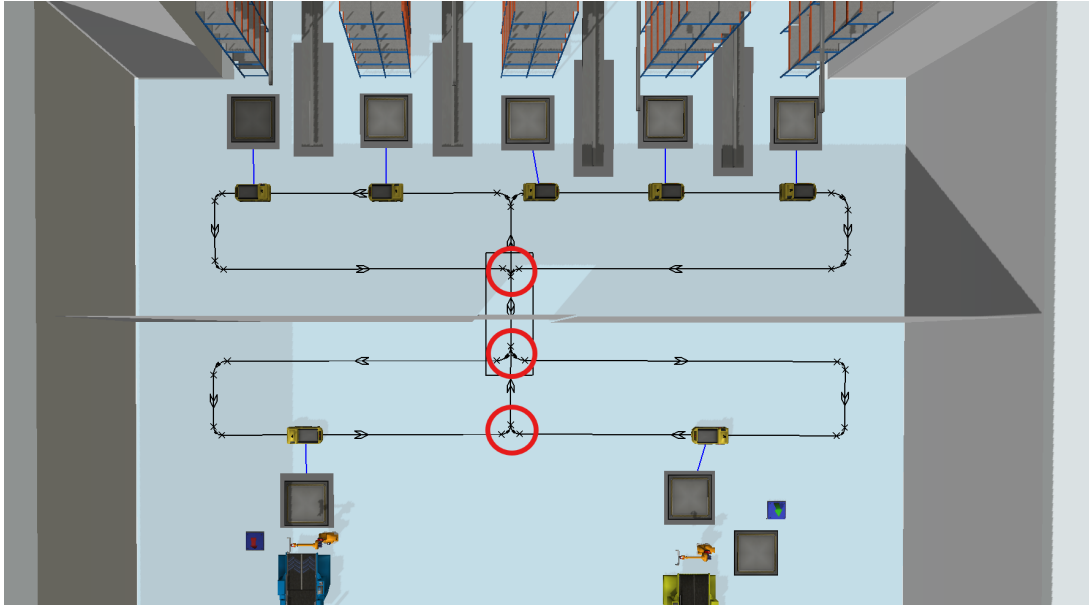


Figura 5.2: Puntos críticos de cruce en la red de AGVs de la sección de fresco.

### 5.3. Aplicación del algoritmo A\*

Este es el mapa de calor, donde se observan los caminos empleados por los AGVs en A\*. Los caminos son muy simples, prácticamente líneas rectas, ya que acuden por el camino más corto de cola a cola para cargar y descargar pallets.

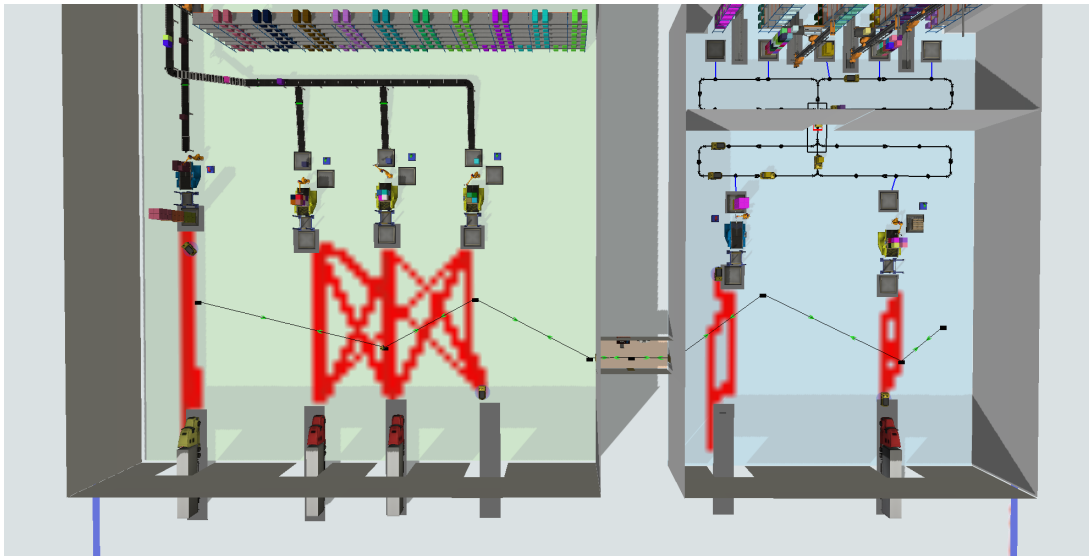


Figura 5.3: Mapa de calor de recorridos de los AGVs que utilizan A\*.

En la planta, el algoritmo A\* se emplea únicamente en el transporte de pallets cola-camión. Los vehículos que usan este algoritmo son los AGVs que se encuentran entre las colas a las que llegan los pallets ya montados (QueuePalletsX) y los camiones de carga

y descarga. En total hay 4 AGVs con este comportamiento: dos de ellos se encargan del transporte del pallet al camión en la sección de seco y fresco respectivamente. Los otros dos se encargan de la función inversa: cogen los pallets del camión que trae las cargas de repuesto y las llevan a QueuePalletsX en fresco y seco respectivamente.

Los AGVs se mueven de forma libre tomando la ruta más corta hasta su destino, ya que se mueven en línea recta. Se han empleado barreras tanto en la zona de las colas como en la zona de los camiones para que los AGVs no atraviesen los elementos y respeten las zonas de seguridad y distancias con los objetos, manteniendo un entorno realista. No se emplean elementos como caminos preferentes, ya que lo que se busca es un recorrido lo más sencillo posible para agilizar el transporte de pallets en distancias cortas.

## 5.4. Agentes Inteligentes utilizados

### 5.4.1. Agente de cruce

El código utilizado para el agente de cruce es el siguiente:

**On entry proximity**

```
/**Custom Code*/
Agent agent = param(1);
Agent neighbor = param(2);
int numInProximity = param(3);

Object agv1 = agent.object;
Object agv2 = neighbor.object;
Object agvToStop;

// Elegir UNO solo (criterio con id)
if (agv1.cruce == 1) { agvToStop = agv1; }
else { agvToStop = agv2; }

agvToStop.stop(1);
await Delay.seconds(0.5);
agvToStop.resume();
```

Este agente gestiona los cruces de los AGVs en el AGV network del fresco. Para ello, se establecen id=1 e id=2 de forma equitativa entre los 7 AGVs. Su funcionamiento se basa en que cuando dos AGVs se acercan, al entrar en proximidad, uno de ellos se para durante

0.5 s y el otro sigue avanzando. Este tiempo de espera se ha calculado en función de los tiempos obtenidos del robot siguelíneas y ajustados al comportamiento del modelo.

Entonces, el código de **On Enter Proximity** detiene uno de los AGVs durante un tiempo determinado mientras el otro avanza. Una vez transcurrido ese tiempo, el AGV reanuda su marcha con **resume** y sigue el curso de la simulación.

#### 5.4.2. Agente de camiones

El código del agente de camiones se divide en dos triggers. **On entry proximity**

```
Agent agent = param(1);
Agent neighbor = param(2);
int numInProximity = param(3);

Object agv1 = agent.object;
Object agv2 = neighbor.object;
Object agvToStop;
Object agvOther;

// Elegir UNO solo (criterio con id)
if (agv1.id > agv2.id) {
    agvToStop = agv1;
    agvOther = agv2;
} else {
    agvToStop = agv2;
    agvOther = agv1;
}

// Reducir velocidad
setvarnum(agvToStop, "maxspeed", 0.5);
agvToStop.stop(0);
agvToStop.resume();
```

#### **On exit proximity**

```
Agent agent = param(1);
Agent neighbor = param(2);
int numInProximity = param(3);
```

```

Object agv1 = agent.object;
Object agv2 = neighbor.object;
Object agvToStop;
Object agvOther;

// Elegir UNO solo (criterio con id)
if (agv1.id > agv2.id) {
    agvToStop = agv1;
    agvOther = agv2;
} else {
    agvToStop = agv2;
    agvOther = agv1;
}

// Poner velocidad original
setvarnum(agvToStop, "maxspeed", 2.0);
agvToStop.stop(0);
agvToStop.resume();

```

Este agente trata de regular la velocidad entre AGVs y camiones, de forma que cuando un AGV esté a una distancia en ratio menor de un camión, su velocidad se reducirá de 2 m/s a 0.5 m/s mientras se encuentre próximo a este.

Para ello, se han asignado como agentes todos los AGVs y los camiones involucrados en el proceso de carga y descarga de pallets. Sin embargo, el comportamiento de proximidad solo se aplica a los AGVs, ya que los vehículos que interesa que se acerquen y se alejen reduciendo la velocidad son los AGVs.

De esta manera, el código se divide en dos triggers: On Enter Proximity y On Exit Proximity. On Enter Proximity se ejecuta al entrar en proximidad ambos vehículos, momento en el que se elegirá al vehículo con mayor id. Previamente se ha asignado id=2 para los AGVs e id=1 para los camiones, de forma que los vehículos que se escojan para disminuir la velocidad siempre sean los AGVs. Entonces se establecerá la velocidad del AGV a 0.5 m/s e inmediatamente se parará y volverá a retomar la marcha. Esto se debe a que en A\*, al principio se precalcula la ruta con los valores que tiene y aunque se cambie el maxSpeed a mitad de ruta, no se ve reflejado en la simulación hasta que vuelve a calcular otra ruta. Por ello, se fuerza a parar y volver a seguir la ruta para recalcularla y que se simule el cambio de velocidad.

En On Exit se ejecuta prácticamente el mismo código: se elige el vehículo con mayor id, que siempre será el AGV, y se le establece la maxspeed a 2 m/s, como estaba originalmente.

Justo después, se realiza el stop y el resume para recalcular la ruta y que se vea en la simulación el cambio de velocidad.

### 5.4.3. Agente supervisor

El código del agente supervisor es el siguiente.

#### On enter proximity

```
/**Custom Code*/
Agent agent = param(1);
Agent neighbor = param(2);
int numInProximity = param(3);

Object agv1 = agent.object;
Object agv2 = neighbor.object;
Object agvToStop;

// Elegir UNO solo (criterio con id)
if (agv1.id == 2) { agvToStop = agv1;}
else { agvToStop = agv2;}

agvToStop.stop(1);
```

#### On exit proximity

```
/**Custom Code*/
Agent agent = param(1);
Agent neighbor = param(2);
int numInProximity = param(3);

Object agv1 = agent.object;
Object agv2 = neighbor.object;
Object agvToStop;

// Elegir UNO solo (criterio con id)
if (agv1.id == 2) { agvToStop = agv1; }
else { agvToStop = agv2; }

agvToStop.resume();
```



Este agente se encarga de mantener la seguridad de un operario de supervisión de la planta. Este operario realiza una ruta por distintos puntos de la planta en los que circulan AGVs. El objetivo de este agente es que, al entrar en proximidad un AGV con el operario, este se pare hasta que el operario se aleje de la zona de proximidad de su ruta. Para ello, en un trigger On Entry Proximity se elige el taskExecutor con mayor id, ya que al operario se le ha puesto id=1 y los AGVs tienen id=2, y este se parará indefinidamente. Es entonces cuando interviene el trigger On Exit Proximity, el cual elige el taskExecutor que se parará con el mismo criterio, siempre el AGV, y cuando salga de la proximidad volverá a reanudar su ruta con resume.

## Conclusiones y Recomendaciones

La propuesta desarrollada demuestra que la automatización integral de una nave logística de alimentación es técnica y funcionalmente viable cuando el diseño se apoya en una correcta segmentación de flujos, una lógica de control coherente y una validación previa mediante simulación.

Desde el punto de vista de la ingeniería industrial, el proyecto confirma varias ideas clave. En primer lugar, la automatización no debe entenderse como una simple sustitución de mano de obra por tecnología, sino como una reorganización completa del sistema productivo. En segundo lugar, el dimensionamiento óptimo de recursos no coincide necesariamente con la máxima dotación disponible; el mejor rendimiento se alcanza cuando almacenamiento, transporte, consolidación y reposición quedan equilibrados. En tercer lugar, la coexistencia de secciones con distinta naturaleza térmica y operativa obliga a diseñar soluciones específicas para cada una, evitando extrapolaciones simplistas.

Los principales resultados del proyecto son los siguientes:

- política de recarga recomendada en seco: activación con 7 pallets;
- política de recarga recomendada en fresco: escenario notable, equivalente a 7 pallets;
- configuración óptima de paletizado en seco: 3 colas y 3 combiners;
- configuración óptima en fresco: 7 AGVs y una línea principal de combiner;
- necesidad de mantener control explícito sobre esclusas, cruces y proximidad entre recursos móviles.

Entre los desafíos más relevantes del desarrollo destacan la coordinación entre subsistemas heterogéneos, la necesidad de sincronizar eventos físicos y lógicos, y la validación de métricas temporales en operaciones de paletizado. Esta última cuestión ha puesto de manifiesto la importancia de diseñar cuidadosamente la captura de datos en modelos de simulación para evitar interpretaciones erróneas.

En ese sentido, uno de los aspectos más útiles del proyecto ha sido comprobar que el rendimiento de una planta automática no depende solo de incorporar tecnología, sino de cómo se relacionan entre sí los distintos recursos. A lo largo del trabajo se ha visto varias

veces que añadir más capacidad no garantiza una mejora. De hecho, en algunos escenarios la sobrecapacidad genera más ineficiencias que beneficios. Esa idea, aunque pueda parecer obvia en teoría, se entiende mucho mejor cuando se ve reflejada en resultados concretos de simulación.

El proyecto integra conocimientos adquiridos en automatización, simulación de sistemas discretos, logística, programación, diseño de procesos y control de vehículos móviles. Como líneas de mejora futuras, se recomienda:

- incorporar modelos energéticos y de consumo de recursos;
- ampliar el sistema de control de calidad de pallets con lógica geométrica más detallada;
- refinar la medición del tiempo de paletizado;
- estudiar políticas dinámicas de recarga dependientes de la demanda instantánea;
- evaluar escenarios de fallo o indisponibilidad parcial de recursos críticos.

En conjunto, la solución propuesta constituye una base sólida para el diseño de una plataforma logística automatizada orientada a la gran distribución alimentaria.

---

# Referencias

1. FlexSim. *Demo Models*. Disponible en: <https://flexsim.es/DEMO-Models/>
2. Mercadona. *Mercadona invierte más de 60 M€ en la optimización y modernización de su nave para frescos del bloque logístico de Riba-roja del Túria*. Disponible en: <https://info.mercadona.es/es/actualidad/mercadona-invierte-mas-de-60-me-en-la-optimizacion-y-modernizacion-de-su-nave-para-frescos-del-bloque-logistico-de-riba-roja-del-turia-valencia/news>
3. Documentación oficial de FlexSim y materiales docentes de la asignatura.
4. Apoyo puntual de herramientas de inteligencia artificial generativa para la revisión de redacción, estructuración técnica y reformulación académica del texto, siempre bajo supervisión, verificación y edición final del equipo autor.

## Anexos

### A.1. Manual de instalación y funcionamiento

Para la correcta ejecución del modelo se requiere que los siguientes archivos se encuentren en la misma carpeta que el ejecutable del modelo `.fsm`:

- `lista_ordenes.py`
- `lista_ordenes_fresco.py`

Ambos scripts generan de forma aleatoria las órdenes de trabajo que alimentan la simulación, evitando la creación manual de pedidos y aumentando el realismo del comportamiento de la planta.

Una vez colocados estos archivos en la misma carpeta del modelo, FlexSim puede invocar sus funciones desde FlexScript durante la ejecución. No es necesario modificar manualmente los pedidos en cada prueba, lo que facilita bastante la repetición de experimentos y la comparación entre escenarios.

### A.2. Descripción del script `lista_ordenes.py`

Este script genera las órdenes de la sección de seco. Define un conjunto de referencias válidas, crea un identificador único por pedido y construye órdenes de 12 líneas sin repetición interna. La selección de referencias está ponderada para favorecer determinados productos según la unidad de su código, reproduciendo así una demanda no uniforme.

Además, el script asigna automáticamente la cola de destino de cada línea, distribuyendo los productos de un pedido entre las tres líneas de paletizado de seco de forma cíclica y controlada.

Esta forma de generación resulta útil porque evita una distribución completamente plana de la demanda. Algunas referencias aparecen con más frecuencia que otras, lo que hace que la simulación se comporte de una forma más creíble y que el sistema de recarga tenga que reaccionar realmente ante consumos desiguales.

A continuación se muestra el código:

```
import random

# Lista de productos de seco
VALID_PRODUCTS = [
    110, 111, 112, 113, 114, 115, 116, 117, 118, 119,
    120, 121, 122, 123, 124, 125, 126, 127, 128, 129,

    210, 211, 212, 213, 214, 215, 216, 217, 218, 219,
    220, 221, 222, 223, 224, 225, 226, 227, 228, 229,

    310, 311, 312, 313, 314, 315, 316, 317, 318, 319,
    320, 321, 322, 323, 324, 325, 326, 327, 328, 329,

    410, 411, 412, 413, 414, 415, 416, 417, 418, 419,
    420, 421, 422, 423, 424, 425, 426, 427, 428, 429,

    510, 511, 512, 513, 514, 515, 516, 517, 518, 519,
    520, 521, 522, 523, 524, 525, 526, 527, 528, 529,

    610, 611, 612, 613, 614, 615, 616, 617, 618, 619,
    620, 621, 622, 623, 624, 625, 626, 627, 628, 629
]

if 'order_counter' not in globals():
    order_counter = 0

# Variables internas para la orden generada
current_product_list = []
current_num_lineas = 0

# Genera ordenes de seco
def nueva_orden_id():
    global order_counter
    order_counter += 1
    return order_counter

# Genera una lista de productos para la orden, con 12 lineas, sin repetidos y pondera
```

```

def nueva_orden_num_lineas():
    global current_product_list, current_num_lineas

    current_num_lineas = 12
    current_product_list = []

    productos_disponibles = VALID_PRODUCTS.copy()
    pesos = []

    for p in productos_disponibles:
        unidad = p % 10
        peso = 10 - unidad
        pesos.append(peso)

    for _ in range(current_num_lineas):
        elegido = random.choices(productos_disponibles, weights=pesos, k=1)[0]

        idx = productos_disponibles.index(elegido)

        current_product_list.append(elegido)

        productos_disponibles.pop(idx)
        pesos.pop(idx)

    return current_num_lineas

# Dado un numero de linea, devuelve el producto correspondiente
def nueva_orden_producto(i):
    i = int(i)
    return current_product_list[i - 1]

# Devuelve la cola destino de la orden
def nueva_orden_queue(i):
    i = int(i)
    num_queue = ((i - 1) % 3) + 1
    return num_queue

# Devuelve la lista de VALID_PRODUCTS para el control de cantidad en almacen

```

```
def valid_number_por_orden(i):
    i = int(i)
    return VALID_PRODUCTS[i - 1]
```

### A.3. Descripción del script `lista_ordenes_fresco.py`

El script de fresco reproduce la misma lógica general, adaptada al catálogo de referencias refrigeradas. También genera pedidos de 12 líneas y utiliza una ponderación específica para modelar un patrón de demanda diferente al de seco. Su integración con FlexSim permite alimentar de forma dinámica la sección refrigerada sin necesidad de definir previamente todas las órdenes.

La diferencia en la ponderación entre seco y fresco tiene interés porque permite simular comportamientos de consumo distintos según la naturaleza del producto. No todos los artículos tienen la misma rotación ni la misma criticidad logística, y trasladar esa diferencia al modelo ayuda a que los resultados sean más representativos.

A continuación se muestra el código:

```
import random

# =====
# GENERAR PRODUCTOS 110-429
# =====
VALID_PRODUCTS = [
    110, 111, 112, 113, 114, 115, 116, 117, 118, 119,
    120, 121, 122, 123, 124, 125, 126, 127, 128, 129,

    210, 211, 212, 213, 214, 215, 216, 217, 218, 219,
    220, 221, 222, 223, 224, 225, 226, 227, 228, 229,

    310, 311, 312, 313, 314, 315, 316, 317, 318, 319,
    320, 321, 322, 323, 324, 325, 326, 327, 328, 329,

    410, 411, 412, 413, 414, 415, 416, 417, 418, 419,
    420, 421, 422, 423, 424, 425, 426, 427, 428, 429
]

# =====

if 'order_counter' not in globals():
```



```

    order_counter = 0

# Variables internas para la orden generada
current_product_list = []
current_num_lineas = 0

def nueva_orden_id_fresco():
    global order_counter
    order_counter += 1
    return order_counter

def nueva_orden_num_lineas_fresco():
    global current_product_list, current_num_lineas

    current_num_lineas = 12

    productos_disponibles = VALID_PRODUCTS.copy()

    # pesos segun la unidad
    # unidad 0 -> peso 1
    # unidad 9 -> peso 10

    pesos = []

    for p in productos_disponibles:
        unidad = p % 10
        peso = unidad + 1
        pesos.append(peso)
        current_product_list = []

    # seleccion sin repetidos pero ponderada
    for _ in range(current_num_lineas):

        elegido = random.choices(
            productos_disponibles,
            weights=pesos,
            k=1

```

```

    )[0]

    idx = productos_disponibles.index(elegido)

    current_product_list.append(elegido)

    productos_disponibles.pop(idx)
    pesos.pop(idx)

    return current_num_lineas

def nueva_orden_producto_fresco(i):
    i = int(i)
    return current_product_list[i - 1]

# Devuelve la lista de VALID_PRODUCTS para el control de cantidad en almac?n
def valid_number_por_orden(i):
    i = int(i)
    return VALID_PRODUCTS[i - 1]

```

## A.4. Integración Python–FlexSim

La comunicación entre Python y FlexSim se realiza mediante llamadas desde FlexScript a funciones externas encargadas de:

- generar el identificador de la orden;
- establecer el número de líneas del pedido;
- devolver la referencia asociada a cada línea;
- proporcionar información auxiliar para control de stock y asignación de cola.

Esta integración aporta flexibilidad, modularidad y facilidad de mantenimiento, al separar la generación de demanda de la lógica visual y operativa del modelo.

Además, esta separación simplifica futuras modificaciones. Si se quisiera cambiar la forma de generar pedidos, ampliar el catálogo de productos o probar una distribución de demanda distinta, no sería necesario rehacer la estructura del *process flow*; bastaría con actuar sobre los scripts de apoyo y sobre las llamadas correspondientes.

---

# Agradecimientos

Queremos expresar nuestro agradecimiento a Alberto Herrero por compartir con nosotros su experiencia profesional en una nave logística de características similares a la diseñada en este proyecto. Sus explicaciones nos ayudaron a comprender mejor la operativa real de este tipo de instalaciones y a plantear una simulación más fiel, además de permitirnos definir objetivos más coherentes y ajustados a la realidad industrial.

También queremos agradecer a Julia Sapiña por su labor docente y su disposición constante para orientar a sus alumnos. Su interés por ayudar y su forma de acompañar el trabajo han sido fundamentales para que pudiéramos construir un proyecto más sólido, mejor estructurado y con un enfoque más riguroso.